CIS I Assignment 1

Pointer Calibration Design, Implementation, and Validation

Fall 2026

Team member name(s):	Zepei Lin, Yue Ge
Student ID(s):	3323BF, 7A3BD5
Contact information:	zlin97@jh.edu, yge29@jh.edu

Academic-integrity statement

I (we) have followed the rules in completing this assignment.

Signature(s): Zepei Lin Yue Ge

Contents

1 Introduction

2 Mathematical Approach
- 2.1 Notation and frame conventions
- 2.2 Marker-body geometry design
- 2.3 Workspace placement design
- 2.4 Measurement model and origins of uncertainty
- 2.5 Rigid point-set registration using SVD
- 2.6 Marker geometry refinement
- 2.7 Sensor-based tip calibration
- 2.8 Single-observation test prediction
- 2.9 Error and covariance

3 Algorithmic Approach
- 3.1 Marker-body and workspace initialization (Code Cells 2–4)
- 3.2 Marker-body refinement (Code Cell 5)
- 3.3 Calibration-observation collection (Code Cell 6)
- 3.4 Least-squares pointer-tip calibration (Code Cells 7–8)
- 3.5 Single-observation tip prediction (Code Cell 9)
- 3.6 Function-level execution flow

4 Program Structure
- 4.1 File and module organization
- 4.2 Program hierarchy and outputs
- 4.3 Student runtime functions
- 4.4 Validation and sensitivity helper functions
- 4.5 Test classes and sensitivity drivers
- 4.6 Program outputs

5 Validation Approach
- 5.1 Step 8 accuracy within 45° and nearby-body size
- 5.2 Unit Test A: marker refinement
- 5.3 Unit Test B: tip calibration
- 5.4 Limitations

6 Result
- 6.1 Marker-refinement result
- 6.2 Calibrated-tip result
- 6.3 Step 8 test result

7 AI Assistance and Use of Tools

1 Introduction

Optical tracking is used in image-guided navigation to locate an instrument from markers attached to its rigid body. The clinically useful point, however, is the pointer tip, which is not directly observed. Its position must be inferred from the marker geometry and the estimated pointer frame. Manufacturing errors, tracker noise, and small frame rotations can therefore produce a substantial tip error, especially when the tip is far from the reference-frame origin.

This assignment requires marker bodies and placements for a pointer, a fixed calibration object, and a nearby reference. The marker coordinates are first refined from repeated tracker observations. The pointer tip is then calibrated from simultaneous pointer/calibration poses and a point-sensor reading. Finally, one simultaneous pointer/nearby observation is used to predict the tip in the nearby frame at each test pose. The required performance is approximately 2 mm when the pointer is within 45° of vertical. The solution uses SVD rigid registration, fixed-batch marker refinement, least-squares tip calibration, and relative-frame prediction; marker geometry and placement are chosen to improve rotational accuracy while remaining inside the tracking volume and avoiding collisions.[1,4]

2 Mathematical Approach

2.1 Notation and frame conventions

Bold lowercase symbols denote three-dimensional column vectors. A star denotes hidden physical ground truth; a hat denotes an estimate. The subscripts m, k, and j denote marker body, marker, and observation, and the parenthesized superscript (r) denotes refinement iteration. Lengths are in millimeters and angles are in radians. A rigid frame acts on a point as

$$F\boldsymbol{x} = R\boldsymbol{x} + \boldsymbol{p}, \qquad F^{-1}\boldsymbol{y} = R^T(\boldsymbol{y} - \boldsymbol{p}), \qquad R^T R = I, \quad \det R = 1. \tag{1}$$

Frames are written as $F = \mathrm{Frame}(R, \boldsymbol{p})$. The translation is the source-frame origin expressed in the destination frame; it is not generally the tip position. Define $\mathrm{sk}(\boldsymbol{u})\boldsymbol{v} = \boldsymbol{u} \times \boldsymbol{v}$.

Table 1: Mathematical symbols and code variables.

Symbol	Meaning	Repository variable
$\boldsymbol{a}_{m,k}$	Nominal marker coordinate in the marker-body frame.	Initial element of `body.nominal_marker_positions`.
$\boldsymbol{a}^*_{m,k}$	Actual marker coordinate in the marker-body frame; $\boldsymbol{a}^*_{m,k} = \boldsymbol{a}_{m,k} + \Delta\boldsymbol{a}_{m,k}$.	`body.actual_marker_positions[k-1].p`.
$\widehat{\boldsymbol{a}}^{(r)}_{m,k}$	Current estimated marker coordinate; $\widehat{\boldsymbol{a}}^{(0)}_{m,k} = \boldsymbol{a}_{m,k}$.	Current element of `body.nominal_marker_positions`.
$\boldsymbol{b}_{m,j,k}$	Fixed measured coordinate of marker k in tracker-frame observation j; it contains the measurement error $\boldsymbol{\epsilon}_{b,m,j,k}$.	Stored element `result.markers[k-1].p`.
F_{tracker}	Nominal tracker-to-workspace pose.	`F_tracker_approx`; `trk.F`.
F^*_{tracker}	Actual tracker-to-workspace pose in one sampling event.	Internal `F_actual`.

Symbol	Meaning	Repository variable
F_{mB}	Nominal marker-body-to-workspace pose.	`F_approx`; `placed.body.world_frame`.
F^*_{mB}	Actual marker-body-to-workspace pose.	`placed.actual_placement`.
$F_{\mathrm{near}B}$	Nominal nearby-reference-body-to-workspace pose.	`F_near_approx`; `near_placed.body.world_frame`.
$F^*_{\mathrm{near}B}$	Actual nearby-reference-body-to-workspace pose.	`near_placed.actual_placement`; validation only.
$\widehat{F}^{(r)}_{m,j}$	Estimated marker-body-to-tracker pose.	`result.frame`.
$\widehat{F}_{\mathrm{ptr},j}$, $\widehat{F}_{\mathrm{cal},j}$	Pointer-to-tracker and calibration-object-to-tracker pose estimates.	`F_ptr_list[j]`, `F_cal_list[j]`.
$\widehat{F}_{\mathrm{ptr,test}}$	Pointer-to-tracker estimate from the single Step-8 observation; the subscript test is introduced here.	`obs["ptr_mb"].frame`.
$\widehat{F}_{\mathrm{near,test}}$	Nearby-reference-body-to-tracker estimate from the same Step-8 observation.	`obs["near_mb"].frame`.
$\widehat{F}_{\mathrm{ptr/cal},j}$	Estimated pointer-to-calibration-object pose; $\widehat{F}^{-1}_{\mathrm{cal},j}\widehat{F}_{\mathrm{ptr},j}$.	Computed from the two frame lists; no supplied variable.
$\boldsymbol{p}_{\mathrm{sensed},j}$	Actual tip coordinate relative to the calibration sensor, returned for a valid reading.	`sensed_tip_list[j]`.
$\boldsymbol{p}_{\mathrm{tip}}$	Nominal tip coordinate in the pointer frame.	`ptr_tip_approx`; `pointer.nominal_tip_position`.
$\boldsymbol{p}^*_{\mathrm{tip}}$	Actual tip coordinate in the pointer frame.	Internal `pointer.get_actual_tip_position()`.
$\widehat{\boldsymbol{p}}_{\mathrm{tip},j}$	Tip-coordinate estimate in the pointer frame from observation j; $\widehat{F}^{-1}_{\mathrm{ptr},j}\widehat{F}_{\mathrm{cal},j}\boldsymbol{p}_{\mathrm{sensed},j}$.	Computed from `F_ptr_list[j]`, `F_cal_list[j]`, and `sensed_tip_list[j]`.
$\delta\boldsymbol{p}_{\mathrm{tip},j}$, $C_{\widehat{p}_{\mathrm{tip},j}}$	Error and covariance of the estimate from observation j.	Derived from the two frame-error covariances and sensor error; no supplied variable.
$\delta\boldsymbol{p}_{\mathrm{sensed},j}$, $C_{\mathrm{sensed},j}$	Calibration-sensor perturbation and covariance; negligible in the assignment.	No supplied variable.
$\widehat{\boldsymbol{p}}_{\mathrm{tip}}$	Final calibrated tip-coordinate estimate; the mean of the N_j values $\widehat{\boldsymbol{p}}_{\mathrm{tip},j}$.	`p_tip_est`.
$\widehat{C}_{\mathrm{tip}}$	Estimated covariance of $\widehat{\boldsymbol{p}}_{\mathrm{tip}}$.	`p_tip_cov`.
$\boldsymbol{p}^*_t$	Actual tip coordinate in the workspace at a test pose.	Computed from validation ground truth.
$\widehat{\boldsymbol{p}}_t$	Predicted tip coordinate in the workspace at a test pose.	Student-computed prediction.
$\widehat{\boldsymbol{p}}_{t,\mathrm{near}}$	Predicted tip coordinate in the nearby reference-body frame at a test pose.	Computed from the two Step-8 frame estimates and `p_tip_est`.
$\boldsymbol{p}^*_{t,\mathrm{near}}$	Actual tip coordinate in the nearby reference-body frame at a test pose.	Computed from actual placements and the actual tip for validation only.

Symbol	Meaning	Repository variable
F_t	Commanded test pointer pose relative to the workspace.	`F_t_nominal`.
F_t^*	Actual test pointer pose relative to the workspace.	`test_placed.actual_placement`.

Table 2: Nominal, estimated, and actual frames.

Frame and direction	Nominal	Estimated	Actual	Relation
Tracker to workspace	F_{tracker}	None specified.	F^*_{tracker}	$F^*_{\text{tracker}} = F_{\text{tracker}}\Delta F_{\text{tracker}}$.
Marker body to workspace	F_{mB}; nearby instance: $F_{\text{near}\,B}$.	None specified.	F^*_{mB}; nearby instance: $F^*_{\text{near}\,B}$.	$F^*_{mB} = \Delta F_{mB} F_{mB}$ for `PlaceMarkerBody`.
Calibration object to workspace	F_{cal}; equivalently F_{mB} for the calibration body.	None specified.	F^*_{cal}	$F^*_{\text{cal}} = F_{\text{cal}}\Delta F_{\text{cal}}$.
Marker body to tracker	$F^{-1}_{\text{tracker}} F_{mB}$.	$\widehat{F}^{(r)}_{m,j}$; Step-8 instances: $\widehat{F}_{\text{ptr,test}}$ and $\widehat{F}_{\text{near,test}}$.	$(F^*_{\text{tracker}})^{-1} F^*_{mB}$.	$(F^*_{\text{tracker}})^{-1} F^*_{mB}$ $= (F_{\text{tracker}}\Delta F_{\text{tracker}})^{-1}$. $\cdot(\Delta F_{mB} F_{mB})$ During refinement, SVD uses the current $\{\widehat{\boldsymbol{a}}^{(r)}_{m,k}\}$ and the fixed observations $\{\boldsymbol{b}_{m,j,k}\}$.
Test pointer to workspace	F_t	No direct full-frame estimate; the single simultaneous observation gives $\widehat{F}_{\text{ptr,test}}$ and $\widehat{F}_{\text{near,test}}$, which are used to compute $\widehat{\boldsymbol{p}}_{t,\text{near}}$.	F_t^*	$F_t^* = F_t \Delta F_t$.

2.2 Marker-body geometry design

Let $\boldsymbol{q}_{m,k} = \boldsymbol{a}_{m,k} - \overline{\boldsymbol{a}}_m$ denote a marker coordinate relative to the body centroid. For a small body rotation $\boldsymbol{\alpha}_m$ and translation $\boldsymbol{\epsilon}_m$, its tracker-frame displacement is

$$\delta \boldsymbol{b}_{m,k} \simeq R \begin{bmatrix} -\operatorname{sk}(\boldsymbol{q}_{m,k}) & I \end{bmatrix} \begin{bmatrix} \boldsymbol{\alpha}_m \\ \boldsymbol{\epsilon}_m \end{bmatrix}. \tag{2}$$

Because $R^T R = I$ and $\sum_k \boldsymbol{q}_{m,k} = 0$, summing the squared displacements gives

$$\sum_k \|\delta \boldsymbol{b}_{m,k}\|_2^2 \simeq \boldsymbol{\alpha}_m^T G_m \boldsymbol{\alpha}_m + N_k \|\boldsymbol{\epsilon}_m\|_2^2, \qquad G_m = \sum_k \left(\|\boldsymbol{q}_{m,k}\|_2^2 I - \boldsymbol{q}_{m,k} \boldsymbol{q}_{m,k}^T \right). \tag{3}$$

Marker geometry affects rotational observability through G_m; translation information depends only on the marker count. The design therefore maximizes $\lambda_{\min}(G_m)$ and keeps $\kappa(G_m) = \lambda_{\max}(G_m)/\lambda_{\min}(G_m)$ near one.

All bodies use the vertices of a triangular bipyramid, scaled by $R_m = \max_k \|\boldsymbol{q}_{m,k}\|_2$:

$$\boxed{\{\boldsymbol{a}_{m,k}\}_{k=1}^{5} = \left\{(R_m, 0, 0), \left(-\frac{R_m}{2}, \pm\frac{\sqrt{3}R_m}{2}, 0\right), (0, 0, \pm R_m)\right\}.} \tag{4}$$

For this geometry,

$$G_m = R_m^2 \,\mathrm{diag}\left(\frac{7}{2}, \frac{7}{2}, 3\right), \qquad \lambda_{\min}(G_m) = 3R_m^2, \qquad \kappa(G_m) = \frac{7}{6}. \tag{5}$$

This familiar polyhedron is a practical near-optimum, not an unconstrained numerical optimum. Any five points satisfying $\|\boldsymbol{q}_{m,k}\|_2 \le R_m$ obey

$$\lambda_{\min}(G_m) \le \frac{\mathrm{tr}(G_m)}{3} = \frac{2}{3}\sum_k \|\boldsymbol{q}_{m,k}\|_2^2 \le \frac{10}{3}R_m^2. \tag{6}$$

The selected bipyramid reaches $3R_m^2$, or 90% of this upper bound, while remaining centered, non-coplanar, and easy to specify.

The three bodies serve different purposes and need not have equal radii. Published pointer and dynamic-reference designs provide practical examples of compact tracked-marker layouts.[6,7] The primary notebook design uses

$$\boxed{R_{\mathrm{ptr}} = 30 \text{ mm}, \qquad R_{\mathrm{cal}} = 30 \text{ mm}, \qquad R_{\mathrm{near}} = 175 \text{ mm}.} \tag{7}$$

The final tip is expressed in the nearby frame. A small nearby rotation error $\delta\theta_{\mathrm{near}}$ therefore produces approximately

$$e_{\mathrm{near}} \approx d_{\mathrm{near-tip}}\,\delta\theta_{\mathrm{near}}, \qquad \delta\theta_{\mathrm{near}} \propto \frac{1}{R_{\mathrm{near}}}. \tag{8}$$

Hence

$$\boxed{e_{\mathrm{near}} \propto \frac{d_{\mathrm{near-tip}}}{R_{\mathrm{near}}}.} \tag{9}$$

The design iteration follows Equation 9. Sensitivity testing showed that the nearby radius is the dominant factor: a sufficiently large radius gives very accurate predictions. However, an oversized nearby body would obstruct personnel in a practical clinical workspace. The design objective is therefore to minimize R_{near} while retaining the required core-region accuracy. Each iteration moves the nearby body as close to the test region as collision constraints allow, reduces its radius, and reruns the full pipeline. A smaller body can be moved closer again, reducing $d_{\mathrm{near-tip}}$ and partly compensating for the reduced radius. The 175 mm radius is used in the primary run; larger candidates are compared in Section 5.1.

The resulting dimensions and notebook coordinates are

Body	R_m [mm]	Marker span $x \times y \times z$ [mm]	Minimum spacing [mm]	$\lambda_{\min}(G_m)$ [mm^2]
Pointer	30	$45.0 \times 52.0 \times 60.0$	42.4	2700
Calibration	30	$45.0 \times 52.0 \times 60.0$	42.4	2700
Nearby reference	175	$262.5 \times 303.1 \times 350.0$	247.5	91875

$$\{\boldsymbol{a}_{\mathrm{ptr},k}\}_{k=1}^{5} = \{[30,0,0]^T, [-15,\pm 15\sqrt{3},0]^T, [0,0,\pm 30]^T\} \text{ mm}, \tag{10}$$

$$\{\boldsymbol{a}_{\mathrm{cal},k}\}_{k=1}^{5} = \{[30,0,0]^T, [-15,\pm 15\sqrt{3},0]^T, [0,0,\pm 30]^T\} \text{ mm}, \tag{11}$$

$$\{\boldsymbol{a}_{\mathrm{near},k}\}_{k=1}^{5} = \{[175,0,0]^T, [-87.5,\pm 87.5\sqrt{3},0]^T, [0,0,\pm 175]^T\} \text{ mm}. \tag{12}$$

The nominal pointer tip remains $\boldsymbol{p}_{\mathrm{tip}} = [0,0,-150]^T$ mm.

Table 3 gives the requested alternatives in terms of the body-specific radius R_m.

Table 3: Marker layouts for four to eight markers.

N_k	Marker coordinates (x,y,z)	Minimum spacing	$\lambda_{\min}(G)$	$\kappa(G)$
4	Regular tetrahedron: $R_m(1,1,1)/\sqrt{3}$, $R_m(1,-1,-1)/\sqrt{3}$, $R_m(-1,1,-1)/\sqrt{3}$, $R_m(-1,-1,1)/\sqrt{3}$	$\sqrt{8/3}R_m$	$\frac{8}{3}R_m^2$	1.00
5	**Triangular bipyramid:** $(R_m,0,0)$**,** $(-R_m/2,\pm\sqrt{3}R_m/2,0)$**,** $(0,0,\pm R_m)$	$\sqrt{2}R_m$	$3R_m^2$	1.17
6	Regular octahedron: $(\pm R_m,0,0)$, $(0,\pm R_m,0)$, $(0,0,\pm R_m)$	$\sqrt{2}R_m$	$4R_m^2$	1.00
7	Pentagonal bipyramid: five vertices $(R_m\cos(2\pi i/5), R_m\sin(2\pi i/5), 0)$, $i=0,\ldots,4$, and two vertices $(0,0,\pm R_m)$	$2R_m\sin(\pi/5)$	$\frac{9}{2}R_m^2$	1.11
8	Cube: all eight vertices $(\pm R_m/\sqrt{3}, \pm R_m/\sqrt{3}, \pm R_m/\sqrt{3})$	$2R_m/\sqrt{3}$	$\frac{16}{3}R_m^2$	1.00

The selected five-point row initializes `ptr_local_positions`, `cal_local_positions`, and `near_local_positions`. The selected 150 mm pointer length initializes `ptr_tip_approx`. It is a design input: the simulator generates the hidden actual tip position around this nominal coordinate. The analytic envelope establishes workspace feasibility; the assigned simulation cases are used only as a validation check.

Figure 1 shows the common polyhedron; each body scales it by its selected radius R_m.

triangular-bipyramid template for pointer, calibration, and nearby bodies

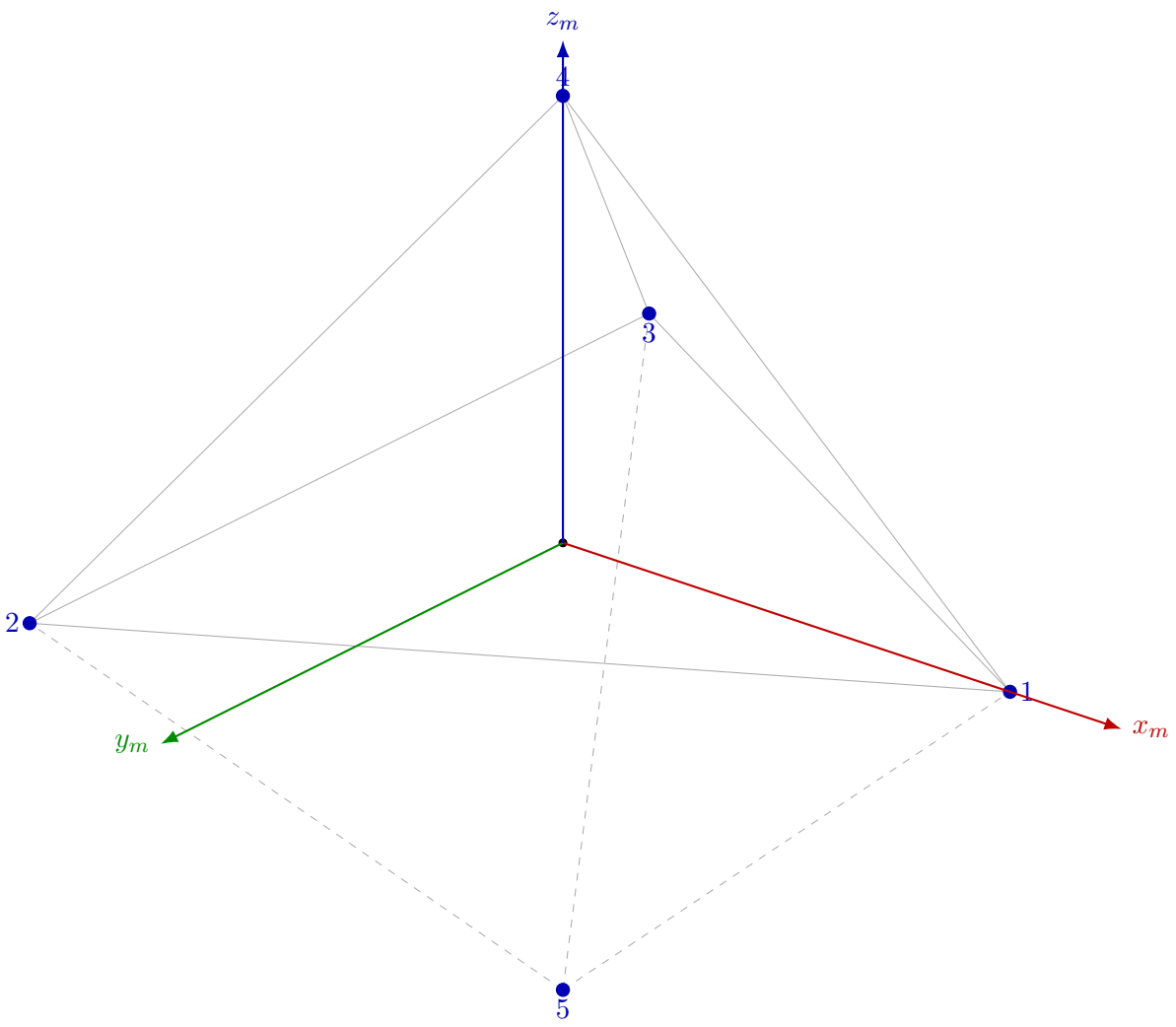


Figure 1: Five-marker triangular bipyramid.

2.3 Workspace placement design

The simulator imposes three relevant placement constraints: every marker must remain in the tracking volume, the nearby marker body must remain outside the pointer test region, and the physical marker spheres must not collide. The simulator has no table or optical-occlusion model. The connecting structure is therefore treated as transparent. The assignment does not prescribe a marker diameter; for collision checks, each marker is modeled as an 11.8 mm sphere, matching a commercially available NDI passive marker.[5]

In tracker coordinates, the assignment tracking volume is modeled as

$$1000 \le y_T \le 2200, \qquad |x_T|, |z_T| \le h(y_T). \tag{13}$$

Here $h(y_T)$ is the available half-width. It increases linearly from 500 mm at the near plane to 1000 mm at the far plane. A 50 mm nominal boundary margin is used. If a body origin is $\boldsymbol{c}$ and every marker satisfies $\|\boldsymbol{a}_{m,k}\|_2 \le R_m$, then every orientation lies inside the radius-R_m ball about

$\boldsymbol{c}$. The following conditions therefore guarantee the margin for all rotations:

$$\boxed{\begin{array}{ll} c_y - R_m \geq 1050, & c_y + R_m \leq 2150, \\ |c_x| + R_m \leq h(c_y - R_m) - 50, & |c_z| + R_m \leq h(c_y - R_m) - 50. \end{array}} \tag{14}$$

This spherical bound is conservative, but it covers continuous orientations without enumerating the assigned Euler-angle grid.

The tracker is first placed in a side-view configuration, and the calibration body is fixed to the right of the test region:

$$\boxed{\begin{array}{ll} {}^W\boldsymbol{p}_T = [300, -1500, 300]^T, & {}^W R_T = I, \\ {}^W\boldsymbol{p}_C = [730, 402, 300]^T, & {}^W R_C = I. \end{array}} \tag{15}$$

A commanded pointer origin is

$${}^W\boldsymbol{p}_P = [300 + t_x, 300 + t_y, 300 + t_z]^T, \qquad {}^T\boldsymbol{p}_P = [t_x, 1800 + t_y, t_z]^T, \tag{16}$$

where $t_x, t_y, t_z \in \{0, \pm 100, \pm 200\}$ mm. With $R_{\text{ptr}} = 30$ mm, all pointer markers lie in $[-230, 230]$ in tracker x and z, and in $[1570, 2030]$ in tracker y. Their minimum frustum clearance is 170 mm.

The nearby placement is selected after choosing its radius. Its y and z coordinates are aligned with the central nominal tip, and it is moved toward the test region until the marker centers retain a 30 mm gap. For radius R_{near}, this gives

$${}^W\boldsymbol{p}_N(R_{\text{near}}) = \left[40 - \frac{R_{\text{near}}}{2},\ 300,\ 150\right]^T, \qquad {}^W R_N = R_z(\pi). \tag{17}$$

This placement rule implements Equation 9: reducing the radius also reduces the body's physical envelope, allowing its origin to move closer to the test region. Radius and position are updated together, and the full pipeline is rerun after each update. For the primary radius $R_{\text{near}} = 175$ mm, the ideal $x_W = -47.5$ mm is rounded outward to -48 mm:

$$\boxed{{}^W\boldsymbol{p}_N = [-48, 300, 150]^T, \qquad {}^W R_N = R_z(\pi).} \tag{18}$$

The final placement is checked directly. The pointer, calibration, and nearby markers remain inside the tracking volume, with minimum boundary clearances of 170.0, 272.0, and 248.4 mm, respectively. The nearby marker spheres remain outside the test region and retain at least 18.7 mm surface clearance from the pointer spheres. During calibration, the pointer and calibration spheres retain at least 73.2 mm surface clearance. Therefore the selected placement has no marker-sphere collisions in the required calibration or test poses.

In Figure 2, the red, orange, and green triangular bipyramids represent the pointer, calibration, and nearby bodies, respectively. The blue box bounds the pointer markers over all Step 8 test poses.

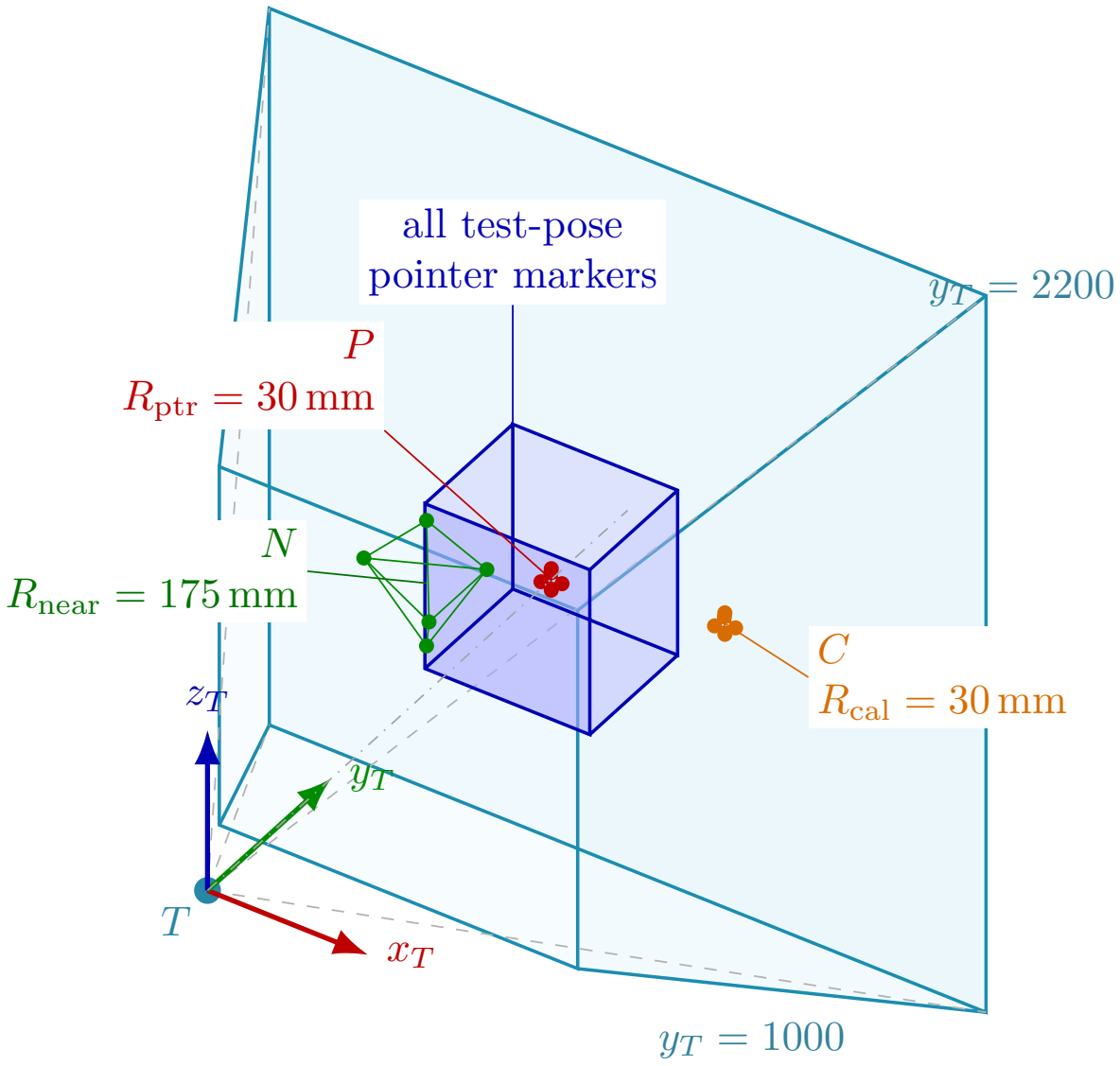


Figure 2: Marker-body placements in tracker coordinates.

In the notebook, ${}^{W}R_T$ and ${}^{W}\boldsymbol{p}_T$ form `F_tracker_approx`, while ${}^{W}R_C, {}^{W}\boldsymbol{p}_C$ and ${}^{W}R_N, {}^{W}\boldsymbol{p}_N$ form the inputs to `simulation.place_marker_body`. Step 8 constructs the pointer frame from `Rot.xyz(rx,ry,rz)` and $[300 + t_x, 300 + t_y, 300 + t_z]^T$.[1,4]

2.4 Measurement model and origins of uncertainty

2.4.1 Error sources and specified limits

Table 4: Error quantities and limits.

Quantity	Bound or model	Value
Marker positions $\Delta\boldsymbol{a}_{m,k}$	$\|\Delta\boldsymbol{a}_{m,k}\|_2 \le \delta_{\text{markers}}$	δ_{markers}; explicitly 2 mm for the pointer and calibration object.

Quantity	Bound or model	Value
Marker body placement ΔF_{mB}	$F^*_{mB} = \Delta F_{mB} F_{mB}$; $\|\boldsymbol{\alpha}_{mB}\|_2 \le \delta_{mBa}$, $\|\boldsymbol{\epsilon}_{mB}\|_2 \le \delta_{mBe}$	δ_{mBa} and δ_{mBe}; numerical values not specified.
Calibration object placement ΔF_{cal}	$\|\boldsymbol{\alpha}_{\text{cal}}\|_2 \le 0.1$; $\|\boldsymbol{\epsilon}_{\text{cal}}\|_2 \le 25\,\text{mm}$	0.1 rad; 25 mm.
Tracker "jiggle" $\Delta F_{\text{tracker}}$	$\|\boldsymbol{\alpha}_{\text{tracker}}\|_2 \le \sigma_{Ta}$; $\|\boldsymbol{\epsilon}_{\text{tracker}}\|_2 \le \sigma_{Tp}$	$\sigma_{Ta} = 0.01\,\text{rad}$; $\sigma_{Tp} = 5\,\text{mm}$.
Marker measurement error $\boldsymbol{\epsilon}_b$	$C_b = \sigma_b^2 I$	Standard deviation $\sigma_b = 0.02\,\text{mm}$; no finite upper bound.
Tracker systematic error	$\text{TrackerSystematicError}(\boldsymbol{b}) = \mathbf{0}$	Zero.
Calibration sensor	Returns the actual relative tip position when $\|\boldsymbol{p}_{\text{sensed}}\|_2 \le 5\,\text{mm}$; otherwise returns $[-10000, -10000, -10000]^T$.	Range 5 mm; no sensor-error term.
Test pose ΔF_t	$F_t^* = F_t \Delta F_t$; ΔF_t is relatively small	No numerical bound specified.
Tip position error $\Delta \boldsymbol{p}_t$	$\|\Delta \boldsymbol{p}_t\|_2 \le 2\,\text{mm}$	Required when the pointer is within 45° of vertical.

2.4.2 Sensor, hand-guiding, and test placement

The calibration sensor returns $\boldsymbol{p}_{\text{sensed},j}$, the actual pointer-tip position relative to the calibration sensor, when

$$\|\boldsymbol{p}_{\text{sensed},j}\|_2 \le 5\,\text{mm}. \tag{19}$$

Otherwise it returns $[-10000, -10000, -10000]^T$. Invalid readings and their paired pose estimates are discarded. No additive sensor-error term is specified.

2.5 Rigid point-set registration using SVD

Let N be the number of corresponding marker pairs used in one frame estimate, with $k = 1, \ldots, N$. The local marker coordinate $\boldsymbol{a}_k$ and measured tracker coordinate $\boldsymbol{b}_k$ refer to the same marker. The goal is to find the body-to-tracker rigid transformation

$$F = \text{Frame}(R, \boldsymbol{p}), \qquad F\boldsymbol{a}_k = R\boldsymbol{a}_k + \boldsymbol{p}, \tag{20}$$

that minimizes the total squared point error:

$$\min_{R^T R = I,\ \det R = 1,\ \boldsymbol{p}} \sum_{k=1}^{N} \|R\boldsymbol{a}_k + \boldsymbol{p} - \boldsymbol{b}_k\|_2^2 . \tag{21}$$

First separate translation from rotation. Define the two centroids and subtract them from the points:

$$\bar{\boldsymbol{a}} = N^{-1} \sum_k \boldsymbol{a}_k, \qquad \bar{\boldsymbol{b}} = N^{-1} \sum_k \boldsymbol{b}_k, \tag{22}$$

$$\widetilde{\boldsymbol{a}}_k = \boldsymbol{a}_k - \bar{\boldsymbol{a}}, \qquad \widetilde{\boldsymbol{b}}_k = \boldsymbol{b}_k - \bar{\boldsymbol{b}}. \tag{23}$$

For any fixed R, the least-squares translation maps the first centroid to the second:

$$\boldsymbol{p} = \bar{\boldsymbol{b}} - R\bar{\boldsymbol{a}}. \tag{24}$$

Substitution removes $\boldsymbol{p}$ from the remaining problem. Using $R^T R = I$, the rotation-dependent objective becomes

$$\sum_{k=1}^{N} \left\| R\widetilde{\boldsymbol{a}}_k - \widetilde{\boldsymbol{b}}_k \right\|_2^2 = \sum_k \|\widetilde{\boldsymbol{a}}_k\|_2^2 + \sum_k \left\|\widetilde{\boldsymbol{b}}_k\right\|_2^2 - 2\sum_k \widetilde{\boldsymbol{b}}_k^T R\widetilde{\boldsymbol{a}}_k. \tag{25}$$

The first two terms do not depend on R. The remaining cross term is collected in the 3×3 matrix

$$H = \sum_{k=1}^{N} \widetilde{\boldsymbol{a}}_k \widetilde{\boldsymbol{b}}_k^T. \tag{26}$$

Equivalently, for $r, s \in \{1, 2, 3\}$,

$$H_{rs} = \sum_{k=1}^{N} \widetilde{a}_{k,r} \widetilde{b}_{k,s}. \tag{27}$$

Using $\boldsymbol{x}^T \boldsymbol{y} = \operatorname{tr}(\boldsymbol{y}\boldsymbol{x}^T)$ and the cyclic property of the trace,

$$\sum_k \widetilde{\boldsymbol{b}}_k^T R\widetilde{\boldsymbol{a}}_k = \operatorname{tr}\left(R \sum_k \widetilde{\boldsymbol{a}}_k \widetilde{\boldsymbol{b}}_k^T \right) = \operatorname{tr}(RH). \tag{28}$$

Therefore,

$$\underset{R^T R = I,\ \det R = 1}{\arg\min} \sum_k \left\| R\widetilde{\boldsymbol{a}}_k - \widetilde{\boldsymbol{b}}_k \right\|_2^2 = \underset{R^T R = I,\ \det R = 1}{\arg\max} \operatorname{tr}(RH). \tag{29}$$

The SVD step begins by decomposing H:

$$H = USV^T, \qquad S = \operatorname{diag}(\sigma_1, \sigma_2, \sigma_3), \qquad \sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0. \tag{30}$$

Writing

$$U = \begin{bmatrix} \boldsymbol{u}_1 & \boldsymbol{u}_2 & \boldsymbol{u}_3 \end{bmatrix}, \qquad V = \begin{bmatrix} \boldsymbol{v}_1 & \boldsymbol{v}_2 & \boldsymbol{v}_3 \end{bmatrix}, \tag{31}$$

the singular vectors satisfy

$$HH^T \boldsymbol{u}_i = \sigma_i^2 \boldsymbol{u}_i, \qquad H^T H \boldsymbol{v}_i = \sigma_i^2 \boldsymbol{v}_i, \qquad i = 1, 2, 3, \tag{32}$$

with $U^T U = I$ and $V^T V = I$. Substitute the SVD of H into the trace objective and use $\operatorname{tr}(ABCD) = \operatorname{tr}(DABC)$:

$$\operatorname{tr}(RH) = \operatorname{tr}(RUSV^T) \tag{33}$$

$$= \operatorname{tr}(V^T RUS). \tag{34}$$

Define

$$W = V^T RU. \tag{35}$$

Because R, U, and V are orthogonal,

$$W^T W = I, \qquad \det(W) = \det(VU^T) \in \{-1, 1\}. \tag{36}$$

The determinant condition is not used to expand the trace. Since S is diagonal, $(WS)_{ii} = W_{ii}\sigma_i$. Therefore,

$$\operatorname{tr}(RH) = \operatorname{tr}(WS) = \sigma_1 W_{11} + \sigma_2 W_{22} + \sigma_3 W_{33}. \tag{37}$$

The determinant condition is used only when maximizing this expression. From $W^TW = I$, each column has unit length:

$$\sum_{r=1}^{3} W_{ri}^2 = 1, \qquad i = 1, 2, 3. \tag{38}$$

Therefore every entry satisfies $|W_{ri}| \leq 1$. Since $\sigma_i \geq 0$,

$$\operatorname{tr}(WS) = \sum_{i=1}^{3} \sigma_i W_{ii} \leq \sum_{i=1}^{3} \sigma_i. \tag{39}$$

The upper bound is reached by $W = I$. Therefore, if $\det(VU^T) = 1$, $W = I$ is admissible and is a maximizer:

$$\operatorname{tr}(WS) = \sigma_1 + \sigma_2 + \sigma_3. \tag{40}$$

If $\det(VU^T) = -1$, $W = I$ is not admissible because $\det(W)$ must also be -1. The determinant-constrained maximum reverses the singular-vector direction associated with the smallest singular value σ_3.

After computing U and V, form the correction matrix

$$D = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(VU^T) \end{bmatrix}. \tag{41}$$

Thus $D = I$ when $\det(VU^T) = 1$, and $D_{33} = -1$ when $\det(VU^T) = -1$. The rotation is then computed directly as

$$\boxed{\widehat{R} = VDU^T.} \tag{42}$$

Use this rotation to recover the translation and body-to-tracker frame:

$$\boxed{\widehat{\boldsymbol{p}} = \bar{\boldsymbol{b}} - \widehat{R}\bar{\boldsymbol{a}}, \qquad \widehat{F} = \operatorname{Frame}(\widehat{R}, \widehat{\boldsymbol{p}}).} \tag{43}$$

Point correspondences must be preserved. At least three noncollinear point pairs are required; the assignment notebook requires at least four noncoplanar markers.

In the repository, `Tracker.procrustes_solver` implements this SVD calculation. `simulation.sample_marker_bodies` calls it using the current `nominal_marker_positions` and the measured marker coordinates, and returns the resulting body-to-tracker frame as `result.frame`.

2.6 Marker geometry refinement

Let marker body m contain N_k markers. Collect N_j tracker observations once and keep their measured coordinates $\boldsymbol{b}_{m,j,k}$ fixed throughout the N_r refinement iterations. The indices are

$$k = 1, \ldots, N_k, \qquad j = 1, \ldots, N_j, \qquad r = 0, \ldots, N_r - 1.$$

The unknown local marker coordinates and the pose associated with each stored observation are fitted to the single least-squares objective

$$\mathcal{J}_m = \sum_{j=1}^{N_j} \sum_{k=1}^{N_k} \|F_{m,j}\boldsymbol{a}_{m,k} - \boldsymbol{b}_{m,j,k}\|_2^2. \tag{44}$$

Initialize the estimated local marker coordinates with the nominal model:

$$\widehat{\boldsymbol{a}}_{m,k}^{(0)} = \boldsymbol{a}_{m,k}. \tag{45}$$

Each iteration alternates between the two groups of unknowns. First, use the current marker model and the same stored observation j to recompute its body-to-tracker frame:

$$\widehat{F}_{m,j}^{(r)} = \arg\min_{F} \sum_{k=1}^{N_k} \left\| F\widehat{\boldsymbol{a}}_{m,k}^{(r)} - \boldsymbol{b}_{m,j,k} \right\|_2^2 = \mathrm{Frame}(\widehat{R}_{m,j}^{(r)}, \widehat{\boldsymbol{p}}_{m,j}^{(r)}). \tag{46}$$

This is the SVD registration from Section 2.5. Second, hold the N_j recomputed frames fixed. The inverse transformation of marker k is

$$(\widehat{F}_{m,j}^{(r)})^{-1}\boldsymbol{b}_{m,j,k} = (\widehat{R}_{m,j}^{(r)})^T \left(\boldsymbol{b}_{m,j,k} - \widehat{\boldsymbol{p}}_{m,j}^{(r)} \right). \tag{47}$$

All N_j inverse-transformed values correspond to the same local marker coordinate. Update marker k by solving

$$\widehat{\boldsymbol{a}}_{m,k}^{(r+1)} = \arg\min_{\boldsymbol{a}} \sum_{j=1}^{N_j} \left\| \boldsymbol{a} - (\widehat{F}_{m,j}^{(r)})^{-1}\boldsymbol{b}_{m,j,k} \right\|_2^2. \tag{48}$$

Differentiating with respect to $\boldsymbol{a}$ gives

$$2\sum_{j=1}^{N_j} \left[\boldsymbol{a} - (\widehat{F}_{m,j}^{(r)})^{-1}\boldsymbol{b}_{m,j,k} \right] = \mathbf{0}. \tag{49}$$

Therefore,

$$\boxed{\begin{aligned} \widehat{\boldsymbol{a}}_{m,k}^{(r+1)} &= \frac{1}{N_j}\sum_{j=1}^{N_j} (\widehat{F}_{m,j}^{(r)})^{-1}\boldsymbol{b}_{m,j,k} \\ &= \frac{1}{N_j}\sum_{j=1}^{N_j} (\widehat{R}_{m,j}^{(r)})^T \left(\boldsymbol{b}_{m,j,k} - \widehat{\boldsymbol{p}}_{m,j}^{(r)} \right). \end{aligned}} \tag{50}$$

Both steps minimize the same fixed-data objective. Therefore the objective cannot increase after either block update:

$$\mathcal{J}_m(\widehat{\boldsymbol{a}}^{(r+1)}, \widehat{F}^{(r)}) \le \mathcal{J}_m(\widehat{\boldsymbol{a}}^{(r)}, \widehat{F}^{(r)}) \le \mathcal{J}_m(\widehat{\boldsymbol{a}}^{(r)}, \widehat{F}^{(r-1)}). \tag{51}$$

After N_r iterations, the output is $\{\widehat{\boldsymbol{a}}_{m,k}^{(N_r)}\}_{k=1}^{N_k}$. As in any joint point-set/pose fit, a common rigid change of the body coordinate system is a gauge freedom; comparisons of marker geometry therefore remove this common rigid alignment.

In the implementation, N_j is `n_readings` and N_r is `passes`. The N_j calls to `simulation.sample_marker_bodies` occur once before the iteration loop, so the acquisition count is N_j, independent of N_r. At iteration r, `Tracker.procrustes_solver` recomputes every $\widehat{F}_{m,j}^{(r)}$ from the current marker model and the stored `result.markers`; the stored `result.frame` is not reused. The fixed measurements are then inverse-transformed, averaged for each marker, and installed together with `placed.body.set_marker_positions`.[1,4]

2.7 Sensor-based tip calibration

For the N_j valid calibration observations, let $j = 1, \ldots, N_j$. In observation j, the pointer has desired orientation $R_{\text{desired},j}$. The quantities used for calibration are the simultaneously estimated poses $\widehat{F}_{\text{ptr},j}$ and $\widehat{F}_{\text{cal},j}$, together with the sensor coordinate $\boldsymbol{p}_{\text{sensed},j}$. An observation is retained only when $\|\boldsymbol{p}_{\text{sensed},j}\|_2 \le 5\,\text{mm}$.[1]

Use both poses from the same observation and define

$$\widehat{F}_{\text{ptr/cal},j} = \widehat{F}_{\text{cal},j}^{-1}\widehat{F}_{\text{ptr},j} = \text{Frame}(\widehat{R}_{\text{ptr/cal},j}, \widehat{\boldsymbol{p}}_{\text{ptr/cal},j}). \tag{52}$$

Its components are

$$\widehat{R}_{\text{ptr/cal},j} = \widehat{R}_{\text{cal},j}^{T}\widehat{R}_{\text{ptr},j}, \tag{53}$$

$$\widehat{\boldsymbol{p}}_{\text{ptr/cal},j} = \widehat{R}_{\text{cal},j}^{T}(\widehat{\boldsymbol{p}}_{\text{ptr},j} - \widehat{\boldsymbol{p}}_{\text{cal},j}). \tag{54}$$

Expressing the pointer tip in the calibration-object frame gives

$$\widehat{R}_{\text{ptr/cal},j}\boldsymbol{p}_{\text{tip}} + \widehat{\boldsymbol{p}}_{\text{ptr/cal},j} \simeq \boldsymbol{p}_{\text{sensed},j}. \tag{55}$$

The equal-weight objective is

$$\widehat{\boldsymbol{p}}_{\text{tip}} = \arg\min_{\boldsymbol{p}} \sum_{j} \left\| \widehat{R}_{\text{ptr/cal},j}\boldsymbol{p} + \widehat{\boldsymbol{p}}_{\text{ptr/cal},j} - \boldsymbol{p}_{\text{sensed},j} \right\|_2^2. \tag{56}$$

This is a linear least-squares problem of the form used for pointing-device calibration in the course notes.[2,3] Stack the equations as

$$A = \begin{bmatrix} \widehat{R}_{\text{ptr/cal},1} \\ \vdots \\ \widehat{R}_{\text{ptr/cal},N_j} \end{bmatrix}, \qquad \boldsymbol{B} = \begin{bmatrix} \boldsymbol{p}_{\text{sensed},1} - \widehat{\boldsymbol{p}}_{\text{ptr/cal},1} \\ \vdots \\ \boldsymbol{p}_{\text{sensed},N_j} - \widehat{\boldsymbol{p}}_{\text{ptr/cal},N_j} \end{bmatrix} \tag{57}$$

$$A\widehat{\boldsymbol{p}}_{\text{tip}} \simeq \boldsymbol{B}, \qquad \widehat{\boldsymbol{p}}_{\text{tip}} = \arg\min_{\boldsymbol{p}} \|A\boldsymbol{p} - \boldsymbol{B}\|_2^2. \tag{58}$$

$$A^{T}A = \sum_{j=1}^{N_j} \widehat{R}_{\text{ptr/cal},j}^{T}\widehat{R}_{\text{ptr/cal},j} = N_j I, \tag{59}$$

$$A^{T}\boldsymbol{B} = \sum_{j=1}^{N_j} \widehat{R}_{\text{ptr/cal},j}^{T}(\boldsymbol{p}_{\text{sensed},j} - \widehat{\boldsymbol{p}}_{\text{ptr/cal},j}). \tag{60}$$

Therefore,

$$\boxed{\widehat{\boldsymbol{p}}_{\text{tip}} = (A^{T}A)^{-1}A^{T}\boldsymbol{B} = \frac{1}{N_j}\sum_{j=1}^{N_j} \widehat{R}_{\text{ptr/cal},j}^{T}(\boldsymbol{p}_{\text{sensed},j} - \widehat{\boldsymbol{p}}_{\text{ptr/cal},j})}. \tag{61}$$

To verify that this least-squares solution is unique and numerically well-conditioned, use

$$\text{rank}(A) = 3, \qquad \sigma_1(A) = \sigma_2(A) = \sigma_3(A) = \sqrt{N_j}. \tag{62}$$

In the implementation, `simulation.hand_guide_pointer` places the pointer for observation j and returns `hg_j`. One call to `simulation.sample_marker_bodies` obtains the paired estimates $\widehat{F}_{\text{ptr},j}$ and $\widehat{F}_{\text{cal},j}$, and `simulation.sense_tip(ptr,hg_j)` obtains $\boldsymbol{p}_{\text{sensed},j}$. An invalid sensor result is discarded together with its two pose estimates. After N_j valid observations have been collected, `numpy.linalg.lstsq` solves the stacked system above. The template function `compute_pivot_calibration` currently accepts only the two pose lists; this method also requires the corresponding list of sensor coordinates.

2.8 Single-observation test prediction

For each test pose, one simultaneous observation provides $\{\boldsymbol{b}_{\mathrm{ptr,test},k}\}$ and $\{\boldsymbol{b}_{\mathrm{near,test},k}\}$ in the tracker frame. For $m \in \{\mathrm{ptr}, \mathrm{near}\}$, SVD with the final marker model gives

$$\begin{aligned} \widehat{F}_{m,\mathrm{test}} &= \mathrm{Frame}(\widehat{R}_{m,\mathrm{test}}, \widehat{\boldsymbol{p}}_{m,\mathrm{test}}), \\ (\widehat{R}_{m,\mathrm{test}}, \widehat{\boldsymbol{p}}_{m,\mathrm{test}}) &= \underset{R^TR=I,\,\det R=1,\,\boldsymbol{p}}{\arg\min} \sum_k \left\| R\widehat{\boldsymbol{a}}_{m,k}^{(N_r)} + \boldsymbol{p} - \boldsymbol{b}_{m,\mathrm{test},k} \right\|_2^2. \end{aligned} \tag{63}$$

The predicted tip coordinate in the nearby-reference-body frame is

$$\boxed{\widehat{\boldsymbol{p}}_{t,\mathrm{near}} = \widehat{F}_{\mathrm{near,test}}^{-1} \widehat{F}_{\mathrm{ptr,test}} \widehat{\boldsymbol{p}}_{\mathrm{tip}}.} \tag{64}$$

The validation value and reported error are

$$\boldsymbol{p}_{t,\mathrm{near}}^{*} = (F_{\mathrm{near}B}^{*})^{-1} F_t^{*} \boldsymbol{p}_{\mathrm{tip}}^{*}, \qquad \delta\boldsymbol{p}_{t,\mathrm{near}} = \boldsymbol{p}_{t,\mathrm{near}}^{*} - \widehat{\boldsymbol{p}}_{t,\mathrm{near}}. \tag{65}$$

In the notebook, one call to `simulation.sample_marker_bodies(trk,[test_placed,near_placed])` provides both frame estimates. The simulator's `test_placed.actual_placement`, `near_placed.actual_placement`, and actual tip coordinate are used only to calculate $\boldsymbol{p}_{t,\mathrm{near}}^{*}$ and $\delta\boldsymbol{p}_{t,\mathrm{near}}$. Each test pose uses one observation.[1,4]

2.9 Error and covariance

2.9.1 Frame estimates from refined marker bodies

After refinement, the actual marker coordinate and the final estimated marker coordinate are related by

$$\boldsymbol{a}_{m,k}^{*} = \widehat{\boldsymbol{a}}_{m,k}^{(N_r)} + \boldsymbol{\epsilon}_{m,k}^{a,(N_r)}. \tag{66}$$

Linearizing the measured marker coordinate about the estimated frame and the final marker model gives

$$\underbrace{\boldsymbol{b}_{m,j,k} - \widehat{F}_{m,j}\widehat{\boldsymbol{a}}_{m,k}^{(N_r)}}_{\boldsymbol{r}_{m,j,k}\ \text{(residual)}} \simeq M_{m,j,k}\boldsymbol{\eta}_{m,j} + \boldsymbol{W}_{m,j,k}. \tag{67}$$

where

$$\boxed{M_{m,j,k} = \widehat{R}_{m,j}\begin{bmatrix} -\mathrm{sk}(\widehat{\boldsymbol{a}}_{m,k}^{(N_r)}) & I \end{bmatrix}, \qquad \boldsymbol{\eta}_{m,j} = \begin{bmatrix} \boldsymbol{\alpha}_{m,j} \\ \boldsymbol{\epsilon}_{m,j} \end{bmatrix}.} \tag{68}$$

The noise term in this linearized frame-estimation equation is

$$\boxed{\boldsymbol{W}_{m,j,k} = \widehat{R}_{m,j}\boldsymbol{\epsilon}_{m,k}^{a,(N_r)} - \boldsymbol{\epsilon}_{b,m,j,k}.} \tag{69}$$

It combines the remaining marker-coordinate error after refinement with the tracker measurement error for marker k. If the marker-model error and tracker measurement error are independent, then

$$C_{W,m,j,k} = \widehat{R}_{m,j}\,\mathrm{Cov}\left(\boldsymbol{\epsilon}_{m,k}^{a,(N_r)}\right)\widehat{R}_{m,j}^{T} + C_b, \qquad C_b = \sigma_b^2 I_3. \tag{70}$$

Stacking all N_k marker equations gives

$$\boldsymbol{r}_{m,j} \simeq M_{m,j}\boldsymbol{\eta}_{m,j} + \boldsymbol{W}_{m,j}, \qquad C_{W,m,j} = \mathrm{Cov}(\boldsymbol{W}_{m,j}). \tag{71}$$

The six entries of $\boldsymbol{\eta}_{m,j}$ are the local rotational and translational correction between the SVD frame estimate and the actual frame. The SVD registration selects the frame that minimizes $\|\boldsymbol{r}_{m,j}\|_2^2$. At this minimum, another linearized pose correction cannot reduce the residual, so

$$M_{m,j}^T \boldsymbol{r}_{m,j} = 0. \tag{72}$$

Substitution of the stacked linearized relation gives

$$M_{m,j}^T \left(M_{m,j}\boldsymbol{\eta}_{m,j} + \boldsymbol{W}_{m,j}\right) = 0, \qquad \boldsymbol{\eta}_{m,j} = -M_{m,j}^{\dagger}\boldsymbol{W}_{m,j}, \tag{73}$$

where, for full-column-rank $M_{m,j}$,

$$M_{m,j}^{\dagger} = \left(M_{m,j}^T M_{m,j}\right)^{-1} M_{m,j}^T. \tag{74}$$

Therefore, the covariance of the pose error after SVD registration is

$$\boxed{C_{\eta_{m,j}} \simeq M_{m,j}^{\dagger} C_{W,m,j} (M_{m,j}^{\dagger})^T.} \tag{75}$$

2.9.2 Tip calibration covariance

For observation j, $\widehat{F}_{\text{cal},j}$ maps the sensed tip from the calibration-object frame to the tracker frame, and $\widehat{F}_{\text{ptr},j}^{-1}$ then maps it to the pointer frame. The resulting single-observation estimate is

$$\boxed{\widehat{\boldsymbol{p}}_{\text{tip},j} = \widehat{F}_{\text{ptr},j}^{-1} \widehat{F}_{\text{cal},j} \boldsymbol{p}_{\text{sensed},j}.} \tag{76}$$

For propagation of the two SVD frame errors, write

$$F_{\text{ptr},j}^{*} = \widehat{F}_{\text{ptr},j} \Delta F_{\text{ptr},j}, \qquad F_{\text{cal},j}^{*} = \widehat{F}_{\text{cal},j} \Delta F_{\text{cal},j}, \tag{77}$$

with

$$\Delta F_{m,j} \simeq \text{Frame}(I + \text{sk}(\boldsymbol{\alpha}_{m,j}), \boldsymbol{\epsilon}_{m,j}), \qquad \boldsymbol{\eta}_{m,j} = \begin{bmatrix} \boldsymbol{\alpha}_{m,j} \\ \boldsymbol{\epsilon}_{m,j} \end{bmatrix}, \quad m \in \{\text{ptr}, \text{cal}\}. \tag{78}$$

Represent the negligible sensor error by

$$C_{\text{sensed},j} = \text{Cov}(\delta \boldsymbol{p}_{\text{sensed},j}) \simeq 0. \tag{79}$$

Substitution into $\boldsymbol{p}_{\text{tip}}^{*} = (F_{\text{ptr},j}^{*})^{-1} F_{\text{cal},j}^{*} (\boldsymbol{p}_{\text{sensed},j} + \delta \boldsymbol{p}_{\text{sensed},j})$ and removal of second-order terms gives

$$\boxed{\begin{aligned} \delta \boldsymbol{p}_{\text{tip},j} &:= \boldsymbol{p}_{\text{tip}}^{*} - \widehat{\boldsymbol{p}}_{\text{tip},j} \\ &\simeq \begin{bmatrix} \text{sk}(\widehat{\boldsymbol{p}}_{\text{tip},j}) & -I \end{bmatrix} \boldsymbol{\eta}_{\text{ptr},j} \\ &\quad + \widehat{R}_{\text{ptr},j}^T \widehat{R}_{\text{cal},j} \begin{bmatrix} -\text{sk}(\boldsymbol{p}_{\text{sensed},j}) & I \end{bmatrix} \boldsymbol{\eta}_{\text{cal},j} \\ &\quad + \widehat{R}_{\text{ptr},j}^T \widehat{R}_{\text{cal},j} \delta \boldsymbol{p}_{\text{sensed},j}. \end{aligned}} \tag{80}$$

If the three error terms are uncorrelated, then

$$\boxed{\begin{aligned} C_{\widehat{p}_{\text{tip},j}} \simeq\ & \begin{bmatrix} \text{sk}(\widehat{\boldsymbol{p}}_{\text{tip},j}) & -I \end{bmatrix} C_{\eta_{\text{ptr},j}} \begin{bmatrix} \text{sk}(\widehat{\boldsymbol{p}}_{\text{tip},j}) & -I \end{bmatrix}^T \\ & + \widehat{R}_{\text{ptr},j}^T \widehat{R}_{\text{cal},j} \begin{bmatrix} -\text{sk}(\boldsymbol{p}_{\text{sensed},j}) & I \end{bmatrix} C_{\eta_{\text{cal},j}} \begin{bmatrix} -\text{sk}(\boldsymbol{p}_{\text{sensed},j}) & I \end{bmatrix}^T \widehat{R}_{\text{cal},j}^T \widehat{R}_{\text{ptr},j} \\ & + \widehat{R}_{\text{ptr},j}^T \widehat{R}_{\text{cal},j} C_{\text{sensed},j} \widehat{R}_{\text{cal},j}^T \widehat{R}_{\text{ptr},j}. \end{aligned}} \tag{81}$$

The final calibrated value is

$$\widehat{\boldsymbol{p}}_{\text{tip}} = \frac{1}{N_j} \sum_{j=1}^{N_j} \widehat{\boldsymbol{p}}_{\text{tip},j}. \tag{82}$$

For independent observations,

$$C_{\widehat{p}_{\text{tip}}} \simeq \frac{1}{N_j^2} \sum_{j=1}^{N_j} C_{\widehat{p}_{\text{tip},j}}. \tag{83}$$

This is the covariance obtained by propagating the per-observation covariances. The supplied simulator does not compute $C_{\widehat{p}_{\text{tip},j}}$, and the template `compute_pivot_calibration` leaves its covariance output for the student to implement. The same final covariance can instead be estimated from the sample variation of the N_j single-observation estimates:

$$\boxed{\widehat{C}_{\text{tip}} = \frac{1}{N_j(N_j-1)} \sum_{j=1}^{N_j} [\widehat{\boldsymbol{p}}_{\text{tip},j} - \widehat{\boldsymbol{p}}_{\text{tip}}] \, [\widehat{\boldsymbol{p}}_{\text{tip},j} - \widehat{\boldsymbol{p}}_{\text{tip}}]^T .} \tag{84}$$

2.9.3 Single-observation test error

Using the error notation from Section 2.9.1, the actual relative tip is

$$\boldsymbol{p}^*_{t,\text{near}} = (\widehat{F}_{\text{near,test}} \Delta F_{\text{near,test}})^{-1} \widehat{F}_{\text{ptr,test}} \Delta F_{\text{ptr,test}} (\widehat{\boldsymbol{p}}_{\text{tip}} + \delta \boldsymbol{p}_{\text{tip}}). \tag{85}$$

Retaining terms linear in the errors gives

$$\boxed{\begin{aligned} \delta \boldsymbol{p}_{t,\text{near}} \simeq\ & \widehat{R}^T_{\text{near,test}} \widehat{R}_{\text{ptr,test}} \delta \boldsymbol{p}_{\text{tip}} \\ & + \widehat{R}^T_{\text{near,test}} \widehat{R}_{\text{ptr,test}} \begin{bmatrix} -\operatorname{sk}(\widehat{\boldsymbol{p}}_{\text{tip}}) & I \end{bmatrix} \boldsymbol{\eta}_{\text{ptr,test}} \\ & + \begin{bmatrix} \operatorname{sk}(\widehat{\boldsymbol{p}}_{t,\text{near}}) & -I \end{bmatrix} \boldsymbol{\eta}_{\text{near,test}}. \end{aligned}} \tag{86}$$

Because both frames are estimated from the same observation, the common tracker pose cancels.

3 Algorithmic Approach

The implementation uses Python 3 in a Jupyter notebook. NumPy is the principal non-standard numerical library; it is used for array operations, linear least squares, singular-value and rank checks, covariance calculations, and numerical validation. Matplotlib is used to generate the sensitivity plots. The Python `unittest` module is part of the standard library and is used for deterministic tests. The course-provided `src` and `data_types` modules supply the simulator and the vector, rotation, frame, and covariance classes used by the implementation.

The computational pipeline proceeds through marker-body initialization and refinement, calibration-observation collection, least-squares tip calibration, and single-observation tip prediction.

3.1 Marker-body and workspace initialization (Code Cells 2–4)

Main tasks. Code Cell 2 defines the three marker-body geometries and nominal pointer tip. Code Cell 3 specifies the tracker, calibration-object, and nearby-reference poses in the workspace. Code Cell 4 constructs and places the bodies, defines the pointer, and creates the tracker.

Table 5: Initialization functions (Cells 2–4).

Code cell	Function	Inputs	Outputs
2	`triangular_bipyramid_positions(radius_mm)`	Marker-body radius in millimeters.	Five local marker coordinates as `vct3` objects.
4	`simulation.define_marker_body(name,positions)`	Body name and nominal local marker coordinates.	A `MarkerBody`.
4	`simulation.place_marker_body(body,frame,precision)`	Marker body, nominal workspace frame, and placement precision.	A `PlacedBody`.
4	`simulation.define_pointer(name,tip,body)`	Pointer name, nominal tip coordinate, and pointer marker body.	A pointer object.
4	`simulation.create_tracker(name,frame)`	Tracker name and nominal workspace frame.	A tracker object.

i. `triangular_bipyramid_positions` generates the nominal local marker coordinates from the selected radius. It implements the five-point construction in Section 2.2:

$$\{\boldsymbol{a}_{m,k}\}_{k=1}^{5} = \{[R_m, 0, 0]^T,\ [-R_m/2, \sqrt{3}R_m/2, 0]^T,\\ [-R_m/2, -\sqrt{3}R_m/2, 0]^T,\ [0, 0, R_m]^T,\ [0, 0, -R_m]^T\}.$$

ii. `simulation.define_marker_body` converts each coordinate list into a marker-body model.

iii. After Code Cell 3 assigns the workspace frames, `simulation.place_marker_body` creates the placed calibration, nearby, and pointer bodies.

iv. `simulation.define_pointer` associates the pointer marker body with its nominal tip coordinate.

v. `simulation.create_tracker` creates the tracker used by all subsequent sampling calls.

3.2 Marker-body refinement (Code Cell 5)

Main task. Code Cell 5 collects 30 observations once for each of the pointer, calibration, and nearby bodies. It then performs three register–average–update passes over the same fixed batch. Thus each body requires 30 tracker samples, independent of the number of refinement passes.

Table 6: Refinement functions (Cell 5).

Code cell	Function	Inputs	Outputs
5	`_point_matrix(points)`	A sequence of `vct3` coordinates.	An $N \times 3$ coordinate array.

Code cell	Function	Inputs	Outputs
5	`_validate_marker_geometry(points)`	Local marker coordinates.	No returned value; rejects an invalid geometry.
5	`simulation.sample_marker_bodies(tracker,placed_bodies)`	Tracker and requested placed bodies.	One `ObservedBody` per body, containing measured markers and an SVD-estimated body-to-tracker pose.
5	`Tracker.procrustes_solver(points,markers)`	Current local marker model and one stored marker observation.	Recomputed body-to-tracker frame for that observation.
5	`_inverse_average_marker_batch(batch,frames)`	Fixed marker observations and their current fitted frames.	Averaged local marker coordinates.
5	`MarkerBody.set_marker_positions(points)`	Refined local marker coordinates.	No returned value; replaces the nominal marker model.
5	`refine_marker_body(trk,placed,n_readings,passes)`	Tracker and placed body; `n_readings=30` is the total fixed-batch size and `passes=3` is the number of optimization iterations.	No returned value; refines the body model in place.

i. `refine_marker_body` collects 30 observations once and controls three refinement passes over that fixed batch.

ii. It first calls `_validate_marker_geometry`; this helper uses `_point_matrix` to check the local coordinate array.

iii. Before the pass loop, `simulation.sample_marker_bodies` is called once for each requested observation, and the measured marker coordinates are stored.

iv. Within each pass, `Tracker.procrustes_solver` recomputes the frame for every stored observation using the current marker model:

$$\widehat{F}_{m,j}^{(r)} = \arg\min_{F} \sum_{k} \left\| F\widehat{\boldsymbol{a}}_{m,k}^{(r)} - \boldsymbol{b}_{m,j,k} \right\|_2^2 .$$

v. `_inverse_average_marker_batch` uses these recomputed frames to transform the fixed measurements back to the body frame and applies

$$\widehat{\boldsymbol{a}}_{m,k}^{(r+1)} = \frac{1}{N_j} \sum_{j=1}^{N_j} (\widehat{F}_{m,j}^{(r)})^{-1} \boldsymbol{b}_{m,j,k}.$$

vi. `MarkerBody.set_marker_positions` installs the averaged coordinates before the next pass begins.

3.3 Calibration-observation collection (Code Cell 6)

Main task. Code Cell 6 collects `N_OBSERVATIONS=60` valid, index-aligned triples of pointer pose, calibration-object pose, and sensed tip position. Each accepted pair of frame estimates comes from one simultaneous tracker sample.

Table 7: Observation-collection functions (Cell 6).

Code cell	Function	Inputs	Outputs
6	`simulation.hand_guide_pointer(pointer,ptr_placed, cal_placed,R_desired)`	Pointer, placed pointer, calibration body, and desired orientation.	The hand-guided pointer placement.
6	`simulation.sample_marker_bodies(tracker,placed_bodies)`	Tracker, pointer body, and calibration body.	Simultaneous pointer and calibration `ObservedBody` values.
6	`simulation.sense_tip(pointer,hand_guided)`	Pointer and its hand-guided placement.	Tip coordinate in the calibration-object frame, or `None`.

i. `simulation.hand_guide_pointer` first places the pointer at the next desired calibration orientation.

ii. `simulation.sample_marker_bodies` then estimates the pointer and calibration-object poses from one simultaneous tracker sample.

iii. `simulation.sense_tip` reads the tip coordinate in the calibration-object frame. The validity condition from Section 2.7 is

$$\|\boldsymbol{p}_{\text{sensed},j}\|_2 \leq 5\,\text{mm}.$$

iv. The two poses and sensed tip are appended together only when the sensor reading is valid; the loop continues until 60 accepted triples have been stored.

3.4 Least-squares pointer-tip calibration (Code Cells 7–8)

Main tasks. Code Cell 7 uses the 60 valid triples collected in Cell 6 in one stacked least-squares solve, producing the fixed tip estimate and its covariance. Code Cell 8 converts the covariance diagonal to per-axis standard deviations and reports the calibrated result. This stage has no iterative passes.

Table 8: Tip-calibration functions (Cells 7–8).

Code cell	Function	Inputs	Outputs
7	`_frame_is_valid(frame,atol)`	A rigid frame and numerical tolerance.	Boolean frame-validity result.
7	`numpy.linalg.lstsq(A,B, rcond=None)`	Stacked rotation blocks and translated sensor vectors.	Tip solution, residual data, rank, and singular values.
7	`compute_pivot_calibration(F_ptr_list,F_cal_list,sensed_tip_list)`	Corresponding pointer frames, calibration frames, and sensed tip coordinates.	Calibrated tip `p_tip_est` and 3×3 covariance `p_tip_cov`.
8	`numpy.sqrt(numpy.clip(diag, 0,None))`	Diagonal of `p_tip_cov`.	Per-axis tip standard deviations.

i. `compute_pivot_calibration` controls the calibration calculation and processes the three aligned input lists.

ii. It calls `_frame_is_valid` for every retained pointer/calibration frame pair.

iii. For each valid observation, `compute_pivot_calibration` first forms the relative frame from Section 2.7,

$$\widehat{F}_{\text{ptr/cal},j} = \widehat{F}_{\text{cal},j}^{-1}\widehat{F}_{\text{ptr},j}.$$

With $N_j = 60$, the observations contribute three equations each, giving $A \in \mathbb{R}^{180\times 3}$ and $\boldsymbol{B} \in \mathbb{R}^{180}$. They are passed once to `numpy.linalg.lstsq`, which computes

$$\widehat{\boldsymbol{p}}_{\text{tip}} = \arg\min_{\boldsymbol{p}} \|A\boldsymbol{p} - \boldsymbol{B}\|_2^2 .$$

iv. The least-squares solution produces `p_tip_est`. The scatter of the single-observation estimates $\widehat{\boldsymbol{p}}_{\text{tip},j} = \widehat{F}_{\text{ptr},j}^{-1}\widehat{F}_{\text{cal},j}\boldsymbol{p}_{\text{sensed},j}$ produces `p_tip_cov` using the Section 2.9 covariance formula

$$\widehat{C}_{\text{tip}} = \frac{1}{N_j(N_j-1)}\sum_{j=1}^{N_j}(\widehat{\boldsymbol{p}}_{\text{tip},j} - \widehat{\boldsymbol{p}}_{\text{tip}})(\widehat{\boldsymbol{p}}_{\text{tip},j} - \widehat{\boldsymbol{p}}_{\text{tip}})^T.$$

v. Code Cell 8 clips the covariance diagonal at zero and applies `numpy.sqrt` to obtain the per-axis standard deviations.

3.5 Single-observation tip prediction (Code Cell 9)

Main task. Code Cell 9 generates 170 representative test poses and predicts the calibrated tip from exactly one simultaneous pointer/nearby tracker sample at each pose.

Table 9: Tip-prediction functions (Cell 9).

Code cell	Function	Inputs	Outputs
9	`simulation.place_marker_body(body,frame,precision)`	Pointer marker body, nominal test frame, and precise placement mode.	The pointer body at one test pose.
9	`simulation.sample_marker_bodies(tracker,placed_bodies)`	Tracker, test pointer body, and nearby body.	One simultaneous pointer/nearby observation used for prediction.
9	`Frame.inv()` and `Frame.__mul__()`	Pointer and nearby frame estimates and calibrated tip.	Predicted tip coordinate in the nearby-reference frame.

i. For each test pose, `simulation.place_marker_body` places the pointer at the nominal pose.

ii. One call to `simulation.sample_marker_bodies` observes the pointer and nearby body simultaneously.

iii. `Frame.inv` and `Frame.__mul__` apply the Section 2.8 single-observation prediction

$$\widehat{\boldsymbol{p}}_{t,\text{near}} = \widehat{F}_{\text{near,test}}^{-1}\widehat{F}_{\text{ptr,test}}\widehat{\boldsymbol{p}}_{\text{tip}}.$$

3.6 Function-level execution flow

Figure 3 shows the implemented call sequence. Every box is a function or method that appears in Code Cells 2–9; arrow labels describe the data passed between calls.

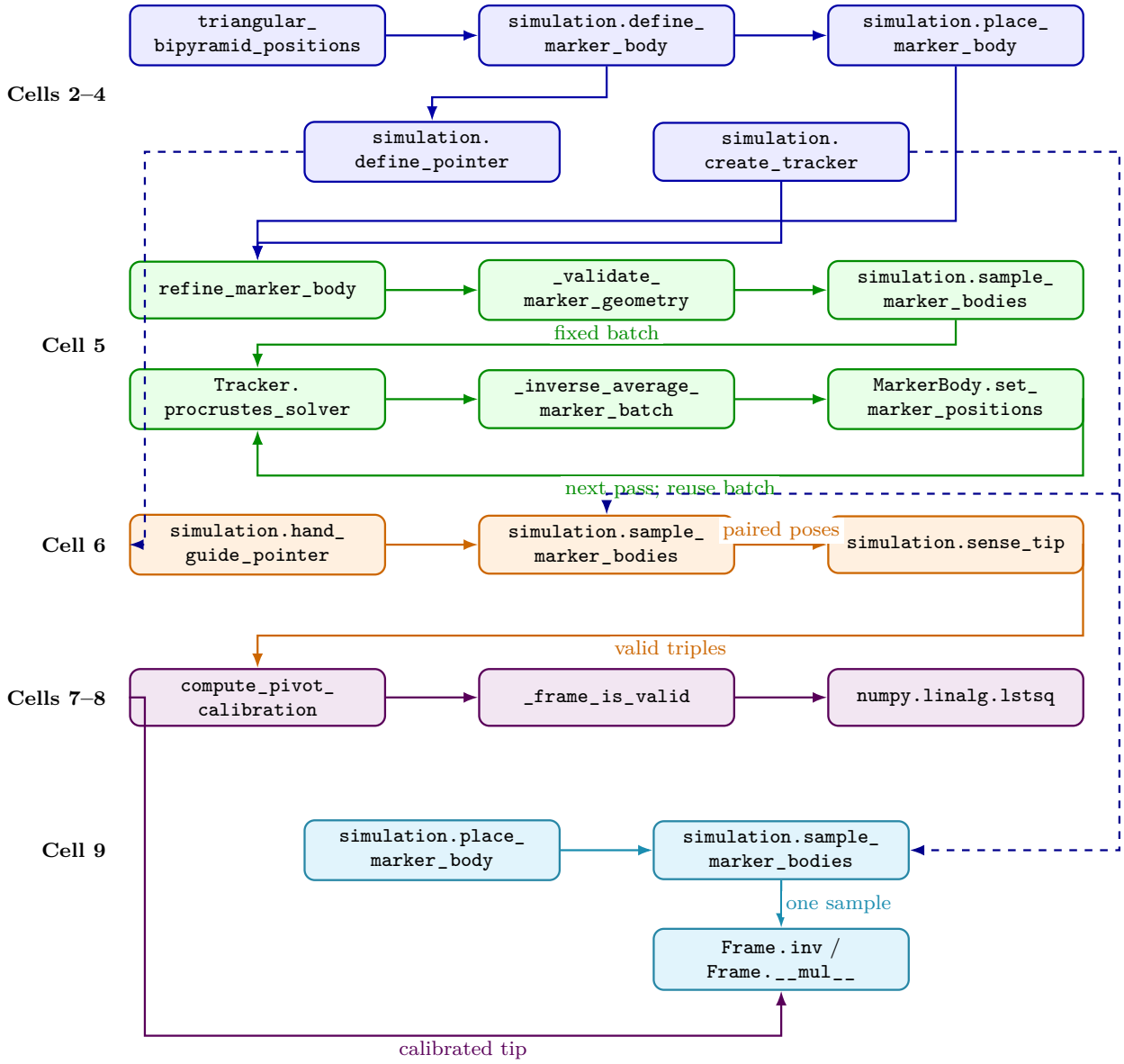


Figure 3: Execution flow (Cells 2–9).

4 Program Structure

The program is implemented in one notebook and uses the supplied `src` modules for simulation and rigid-body operations. The organization of the runtime, test, and sensitivity branches is summarized below.

4.1 File and module organization

Table 10: Program files and modules.

Location	Responsibility
`notebooks/assignment1_template.ipynb`	Defines the marker layouts, workspace poses, refinement and calibration functions, Step 8 prediction loop, validation statistics, unit tests, and sensitivity studies.
`src/__init__.py`	Exposes the assignment API used by the notebook, including `simulation`, `vct3`, `Rot`, and `Frame`.
`src/simulation/`	Provides assignment-facing operations for defining and placing bodies, creating the tracker, hand-guiding the pointer, sensing the tip, and sampling marker bodies.
`src/marker.py` and `src/pointer.py`	Provide the `MarkerBody` and `Pointer` models and maintain nominal and hidden actual geometry.
`src/tracker.py`	Provides noisy marker acquisition and `Tracker.procrustes_solver`, the SVD rigid-registration routine used by sampling and validation helpers.
`src/utils.py`	Provides covariance-controlled random sampling used by the simulator.
`data_types/`	Provides nominal and uncertain vectors, rotations, frames, and covariance types.

4.2 Program hierarchy and outputs

Figure 4 organizes the notebook by purpose and identifies the result produced by each branch. Detailed function interfaces and call relationships are given in Tables 11–13.

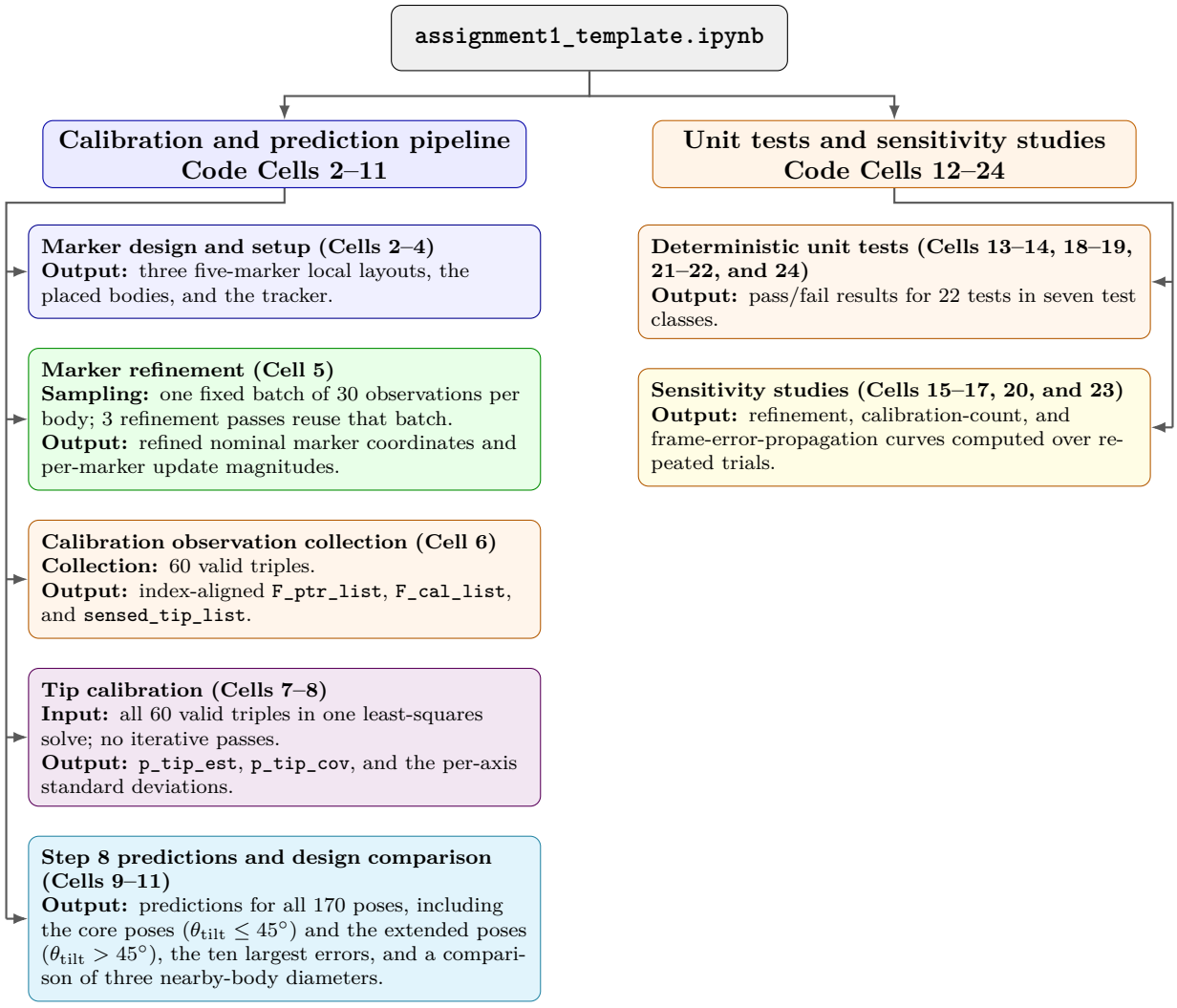


Figure 4: Program hierarchy and outputs.

4.3 Student runtime functions

Table 11 lists every function defined by the notebook before validation begins. A function with no returned value updates an object in place or raises a documented exception when its input contract is violated.

Table 11: Runtime functions.

Cell	Function	Inputs	Output and purpose
2	`triangular_bipyramid_positions(radius_mm)`	Marker-body radius.	Returns five centered, noncoplanar `vct3` coordinates used for the pointer, calibration, and nearby bodies.
5	`_point_matrix(points)`	Sequence of `vct3` coordinates.	Returns an $N \times 3$ floating-point array. It is reused by geometry validation, refinement reporting, and unit tests.

Cell	Function	Inputs	Output and purpose
5	`_validate_marker_geometry(points)`	Local marker-coordinate sequence.	Returns no value; rejects fewer than four points, nonfinite coordinates, or coplanar geometry. It calls `_point_matrix`.
5	`_inverse_average_marker_batch(batch, frames)`	Fixed `ObservedBody` values and their frames recomputed from the current marker model.	Returns one averaged local `vct3` per marker after inverse-transforming every fixed observation; rejects empty or inconsistent batches.
5	`refine_marker_body(trk,placed,n_readings,passes)`	Tracker, placed body, fixed-batch observation count, and pass count.	Returns no value; samples once, recomputes every observation frame in each pass, averages the fixed batch, and updates the `MarkerBody` in place.
7	`_frame_is_valid(frame, atol)`	Candidate `Frame` and tolerance.	Returns a Boolean indicating a finite translation and a finite proper rotation.
7	`compute_pivot_calibration(F_ptr_list,F_cal_list, sensed_tip_list)`	Three index-aligned lists of pointer frames, calibration frames, and sensor readings.	Returns `p_tip_est` and the 3×3 covariance `p_tip_cov`; it filters unusable readings, calls `_frame_is_valid`, constructs the stacked system, and calls `numpy.linalg.lstsq`.

4.4 Validation and sensitivity helper functions

The nearby-design evaluator is defined in Code Cell 11, and the shared unit-test and sensitivity helpers are defined in Code Cell 12. They use isolated test objects or restore the NumPy random state after execution, so they do not alter the calibrated model or the primary Step 8 result.

Table 12: Validation and sensitivity helpers.

Function	Inputs	Output	Principal caller or purpose
`_evaluate_near_design`	Diameter, workspace x coordinate, and setup seed.	All-pose and core-region Step 8 statistics.	Rebuilds, refines, and evaluates one nearby-body candidate while replaying the same Step 8 random sequence.
`_run_unit_test`, `_print_pass_summary`	A `TestCase`, failure message, title, and check labels.	Test result and explicit PASS table.	Execute each deterministic test class and stop on failure.
`_point_rows`, `_unit_marker_observation`	Local points and a known frame.	Coordinate array or exact synthetic marker observation.	Small adapters for refinement tests.
`_exact_refinement_result`	Reading count and pass count.	True/refined points and acquisition/registration call counts.	Verifies exact recovery and fixed-batch reuse.

Function	Inputs	Output	Principal caller or purpose
`_synthetic_calibration_case`	No input.	Known tip, paired frames, and in-range sensed tips.	Supplies exact calibration fixtures.
`_marker_layout_for_count`	Marker count and radius.	Centered 4–8 marker layout.	Builds the tetrahedron, triangular bipyramid, octahedron, pentagonal bipyramid, or cube used in Cell 16.
`_aligned_marker_rmse`	Estimated points, actual points, and tracker.	Gauge-aligned marker-position RMSE.	Removes a common rigid gauge before scoring the refined model.
`_test_marker_cases`, `_test_frame_rmse`	Actual points, seed, model, tracker, and test cases.	Independent noisy observations and test-frame rotation/translation RMSE.	Evaluate a refined model on observations not used for training.
`_refinement_trial`	Layout, reading count, pass count, and seed.	Aligned marker, test-rotation, and test-translation errors.	Runs one isolated fixed-batch refinement trial.
`_refinement_objective`, `_refinement_trace`	Marker model, fixed batch, frames, controls, and seed.	Training residual and per-iteration validation histories.	Check descent and convergence on one continuous optimization trajectory.
`_summarize_trials`	Parameter values, trial callback, and seeds.	Trial array, mean, and sample standard deviation.	Shared repeated-trial aggregator.
`_calibration_truth`, `_perturb_frame`	Observation count, random generator, and noise scales.	Exact calibration data and perturbed frames.	Controlled fixtures for calibration sensitivity.
`_frame_noise_tip_trial`	Observation count and seed.	Calibrated-tip error.	Tests averaging of fixed-strength independent frame errors.
`_rotation_error_angle`, `_frame_error_to_tip_trial`	Estimated/true frames, error scale, seed, and error kind.	Frame-error RMSE and calibrated-tip error.	Propagates rotation or translation error into tip calibration.

4.5 Test classes and sensitivity drivers

Table 13: Unit-test classes and sensitivity drivers.

Cells	Class or driver	Tested behavior and output
13	`TestExactMarkerRecovery`	Checks exact recovery of five marker coordinates and verifies that 4 observations with 3 passes cause 4 acquisitions and 12 SVD registrations.

Cells	Class or driver	Tested behavior and output
14	`TestRefinementInputValidation`	Checks empty and undersized batches, inconsistent observations, coplanar or nonfinite geometry, and invalid refinement controls.
15	Refinement observation/convergence drivers	Sweeps 1–40 fixed-batch observations and traces iterations 0–8 while checking that the training objective does not increase.
16	Marker-count sensitivity driver	Compares the 4-, 5-, 6-, 7-, and 8-marker polyhedra at a common 30 mm radius.
17	Marker-radius sensitivity driver	Compares radii from 30 to 250 mm for the five-marker triangular bipyramid.
18	`TestExactTipRecovery`	Checks exact recovery of a known pointer tip.
19	`TestCalibrationInputValidation`	Checks list alignment, frame type and rigidity, finite data, and sensor type.
20	Calibration-count sensitivity driver	Sweeps 2–60 valid observations with fixed-strength independent frame errors and a $1/\sqrt{N}$ reference.
21	`TestInvalidSensorFiltering`	Checks the assignment sentinel, the valid 5 mm boundary, and missing/nonfinite readings.
22	`TestMinimumCalibration Observations`	Checks rejection of zero or one valid reading and acceptance of two valid readings.
23	Frame-error-propagation drivers	Independently sweep upstream frame rotation and translation errors and report their effect on calibrated-tip error.
24	`TestCalibrationCovariance`	Checks zero covariance for identical estimates and the analytic covariance of a three-sample mean.

4.6 Program outputs

The notebook reports four groups of results:

- **Refined geometry:** updated local marker coordinates for the pointer, calibration, and nearby bodies.
- **Tip calibration:** `p_tip_est`, its covariance `p_tip_cov`, and the standard deviation along each pointer-frame axis.
- **Step 8 performance:** single-observation predictions and error statistics for the poses whose actual pointer-axis tilt satisfies $\theta_{\text{tilt}} \leq 45°$.
- **Validation:** nearby-radius comparisons, 22 deterministic unit-test results, and the refinement and calibration sensitivity plots.

5 Validation Approach

Validation first measures the final Step 8 prediction error over the test poses and compares nearby-body sizes. It then tests marker refinement and tip calibration separately using exact cases, invalid inputs, and sensitivity studies. Hidden simulator values are used only to score completed estimates.

5.1 Step 8 accuracy within 45° and nearby-body size

The nominal Euler-angle components are selected from the assignment set

$$\Theta = \{0, \pm\pi/6, \pm\pi/3, \pm\pi/2\}, \qquad \theta_j \in \Theta. \tag{87}$$

For each evaluated pose, the code computes the actual angle θ_{tilt} between the pointer axis and workspace vertical. Only the $N_{45} = 102$ poses satisfying $\theta_{\text{tilt}} \leq 45°$ are used for the required performance result. This classification uses the transformed pointer axis rather than any single Euler-angle component.

Step 8 predicts a different nearby-frame tip $\widehat{\boldsymbol{p}}_{t,\text{near},i}$ for each pose from one simultaneous pointer/nearby observation. Its error is

$$\Delta\boldsymbol{p}_i = \widehat{\boldsymbol{p}}_{t,\text{near},i} - \boldsymbol{p}^*_{t,\text{near},i}, \qquad e_i = \|\Delta\boldsymbol{p}_i\|_2 . \tag{88}$$

Over the N_{45} required poses, the reported scalar measures are

$$\text{RMSE}_{45} = \sqrt{\frac{1}{N_{45}}\sum_i e_i^2}, \qquad \rho_{2\text{mm},45} = \frac{1}{N_{45}}\sum_i \mathbf{1}(e_i \leq 2\,\text{mm}). \tag{89}$$

The nearby-frame error mean and sample covariance are

$$\overline{\Delta\boldsymbol{p}}_{45} = \frac{1}{N_{45}}\sum_i \Delta\boldsymbol{p}_i, \qquad \widehat{C}_{\Delta p,45} = \frac{1}{N_{45}-1}\sum_i (\Delta\boldsymbol{p}_i - \overline{\Delta\boldsymbol{p}}_{45})(\Delta\boldsymbol{p}_i - \overline{\Delta\boldsymbol{p}}_{45})^T. \tag{90}$$

The code evaluates these with the vector mean and `numpy.cov(...,ddof=1)`. This covariance describes empirical error across different test poses, not repeated measurements at one pose.

For the 350 mm-diameter nearby body,

$$\overline{\Delta\boldsymbol{p}}_{45} = [0.044, -0.546, 0.998]^T \text{ mm}, \qquad \widehat{C}_{\Delta p,45} = \begin{bmatrix} 0.203 & -0.036 & -0.049 \\ -0.036 & 0.170 & -0.109 \\ -0.049 & -0.109 & 0.100 \end{bmatrix} \text{ mm}^2. \tag{91}$$

Table 14: Step 8 accuracy within 45° of vertical.

N_{45}	Mean [mm]	RMSE [mm]	Maximum [mm]	Within 2 mm
102	1.260	1.329	2.155	96.1%

The required poses pass the chosen 95% empirical criterion, although one observed error reaches 2.155 mm.

The nearby body strongly affects this error because its rotation acts on the tip expressed in the nearby frame. As its size increases, the nearby body is placed slightly farther from the test workspace to preserve collision clearance. In Table 15, x_W denotes the x coordinate of the nearby frame's origin in the world coordinate system. The comparison therefore includes both the larger marker-body size and the corresponding placement adjustment.

Table 15: Nearby-body comparison within 45° of vertical.

Diameter [mm]	Radius [mm]	x_W [mm]	RMSE [mm]	Maximum [mm]	Within 2 mm
350	175	−48	1.329	2.155	96.1%
400	200	−60	1.235	2.284	97.1%
500	250	−85	1.022	1.890	100.0%

5.2 Unit Test A: marker refinement

Unit Test A evaluates the `refine_marker_body` function, which updates local marker coordinates from a fixed batch of tracker observations. Tests A.1–A.5 assess coordinate recovery, invalid-input handling, convergence, and sensitivity to the observation count, marker count, and marker-body radius. For the marker-error metric, the refined model is first SVD-aligned to the true model to remove differences in rigid coordinate-frame placement:

$$e_{\text{marker}} = \sqrt{\frac{1}{N_k} \sum_{k=1}^{N_k} \left\| F_{\text{align}} \widehat{\boldsymbol{a}}_k - \boldsymbol{a}_k^* \right\|_2^2}. \tag{92}$$

A.1 Exact marker-coordinate recovery

Four exact observations of a known five-marker body recover all coordinates with a maximum error of 3.949×10^{-14} mm. The initial model is already exact in this deterministic case. Call counts confirm four tracker acquisitions and 12 SVD registrations over three passes, verifying that the same observations are reused and frames are recomputed each pass.

A.2 Invalid refinement inputs

All six invalid-input tests pass. The implementation rejects an empty observation batch, fewer than four observed markers, inconsistent marker/frame structure, coplanar geometry, nonfinite geometry, and invalid reading/pass controls. Each case must raise `ValueError` rather than return a misleading refined model.

A.3 Observation-count and iteration sensitivity

A.3.1 Observation-count sensitivity

Figure 5 shows that a larger fixed batch reduces aligned marker-coordinate RMSE from 0.02521 to 0.00395 mm as the observation count rises from 1 to 40. The independent test-frame errors change little in these trials; the improvement in local marker geometry does not produce a comparable decrease in single-observation frame error. Error bars in the refinement figures are sample standard deviations across eight seeds, not confidence intervals.

The five-marker triangular bipyramid has radius 30 mm and is refined for three passes. Each plotted point is the eight-seed mean of the marker RMSE after alignment to the true geometry.

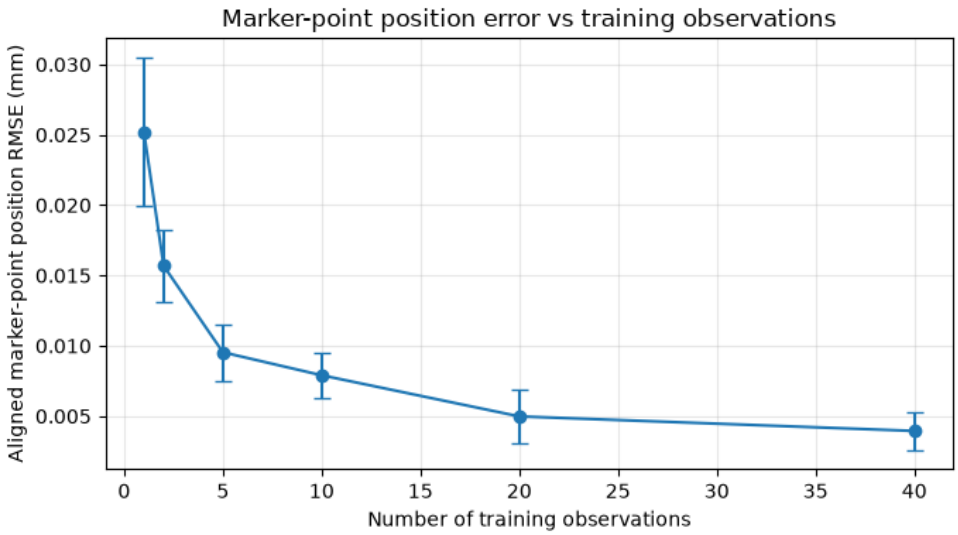


Figure 5: Marker error versus observation count (A.3.1).

A.3.2 Iteration convergence

The convergence test reuses one ten-observation batch. The first pass reduces the training residual from 1.718717 to 0.025804 mm and marker RMSE from 1.718247 to 0.007902 mm. The largest later residual increase is 1.651×10^{-14} mm, consistent with roundoff. Figure 6 displays this rapid initial decrease on a logarithmic scale. The residual is nearly unchanged after the second update, supporting three passes for this tested configuration. Additional passes reuse the same observations and should not be interpreted as additional independent measurements.

Each trial starts from a perturbed marker model and performs eight updates; iteration zero denotes the initial model. The plotted quantity is the mean training marker-residual RMS across eight seeds, with one sample standard deviation shown by the error bars.

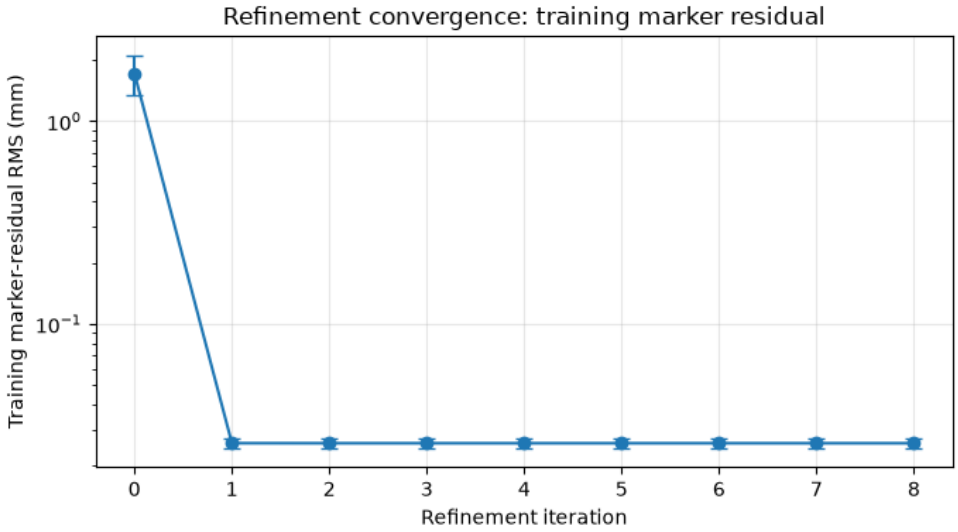


Figure 6: Marker-refinement convergence (A.3.2).

A.4 Marker-count sensitivity

Figure 7 shows the marker-count and layout comparison. In the tested designs, the mean rotation RMSE decreases from 0.03736° to 0.02785° and translation RMSE from 0.01665 to 0.01225 mm. Both marker count and topology change, so this is a design comparison rather than an isolated causal effect of marker count.

Four through eight markers use tetrahedral, triangular-bipyramid, octahedral, pentagonal-bipyramid, and cubic layouts, respectively, with a common maximum radius of 30 mm. Refinement uses ten observations and three passes. The left and right panels show independent test-frame rotation and translation RMSE; points and error bars represent the mean and one sample standard deviation across eight seeds.

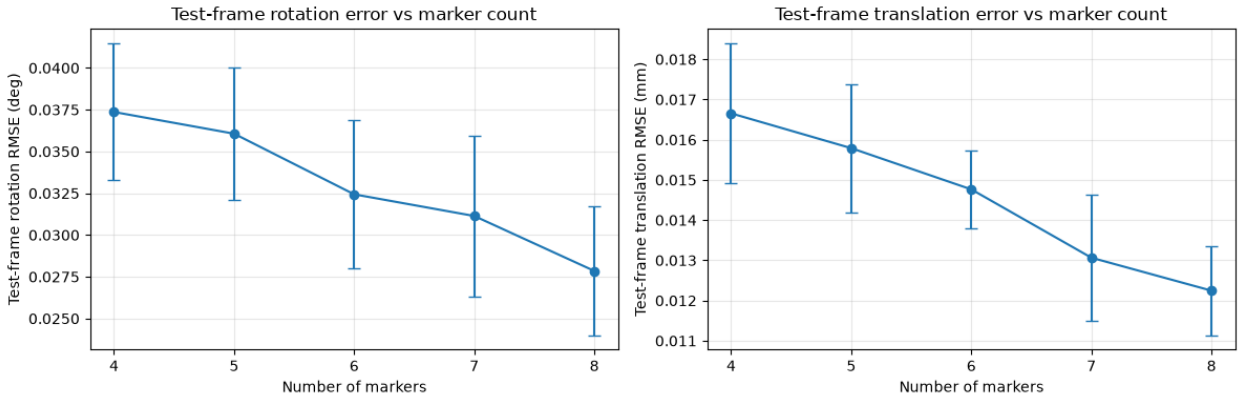


Figure 7: Frame error versus marker count and layout (A.4).

A.5 Marker-radius sensitivity

For the fixed five-marker topology, Figure 8 shows rotation RMSE decreasing from 0.03606° at 30 mm to 0.00435° at 250 mm, approximately following $1/R_m$. Translation RMSE remains approximately 0.01579 mm in these trials. This supports the rotational benefit of spreading the markers without implying that radius alone removes every source of tip-position error.

The triangular-bipyramid topology, ten training observations, and three refinement passes remain fixed. Points and error bars show the mean and one sample standard deviation across eight seeds. Only test-frame rotation RMSE is plotted; translation and marker-coordinate errors are not shown.

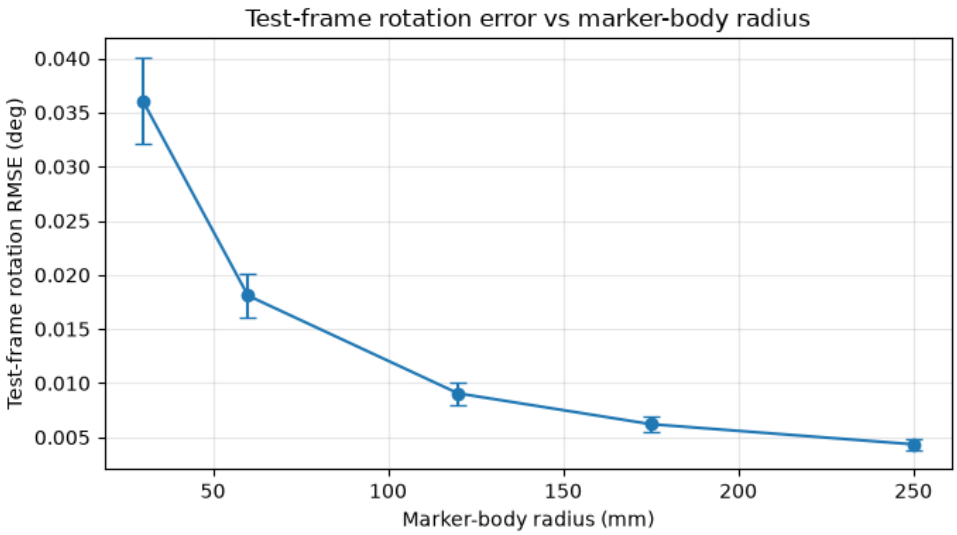


Figure 8: Rotation error versus marker-body radius (A.5).

5.3 Unit Test B: tip calibration

Unit Test B evaluates the `compute_pivot_calibration` function, which estimates the pointer-tip coordinates and their covariance from paired frames and sensor readings. Tests B.1–B.7 assess exact recovery, invalid-input handling, observation-count sensitivity, sensor filtering, the minimum valid sample count, frame-error propagation, and covariance calculation. Calibration error is measured as

$$e_{\text{tip}} = \left\| \widehat{\boldsymbol{p}}_{\text{tip}} - \boldsymbol{p}^*_{\text{tip}} \right\|_2 . \tag{93}$$

B.1 Exact synthetic tip recovery

Exact paired frames and exact sensor readings recover the known tip $[1.5, -0.75, -149]^T$ mm to below 10^{-14} mm per axis. This deterministic case checks transform direction and least-squares recovery without measurement noise.

B.2 Invalid calibration inputs

All six malformed-input tests raise `ValueError`: mismatched list lengths, non-`Frame` inputs, nonorthogonal rotations, singular rotations, nonfinite frames, and invalid sensor types. These checks distinguish interface errors from physically unusable sensor readings.

B.3 Calibration observation-count sensitivity

Figure 9 shows mean tip error falling from 1.32716 to 0.24062 mm as the valid observation count rises from 2 to 60. The trend is approximately consistent with the normalized $1/\sqrt{N}$ reference under these independent frame perturbations. Error bars show the sample standard deviation across 40 seeds; they are not confidence intervals and do not establish a universal error bound.

Exact sensor readings are combined with independently perturbed pointer and calibration frames at a fixed perturbation strength. Points show the mean across 40 seeds. The dashed $1/\sqrt{N}$ reference is normalized to the first measured mean and serves only as a descriptive comparison.

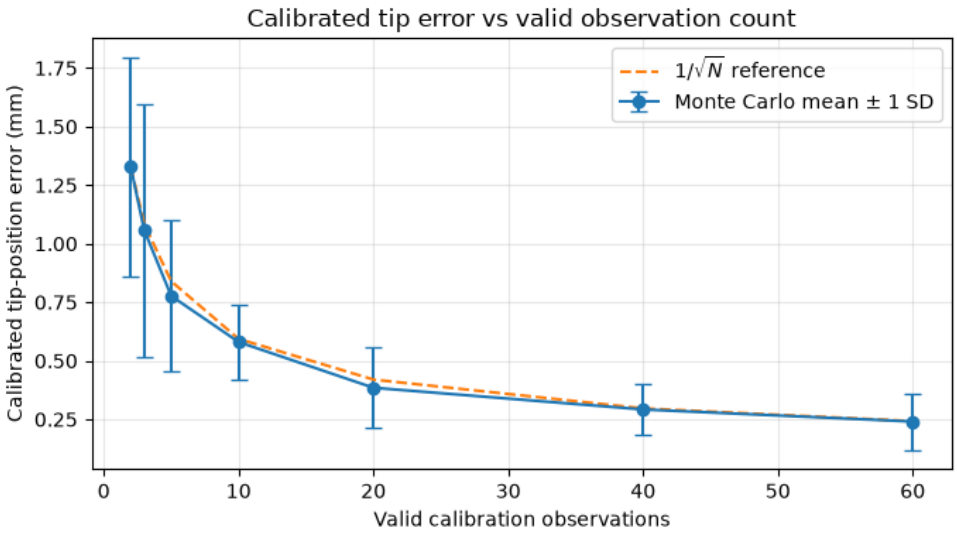


Figure 9: Tip error versus calibration observation count (B.3).

B.4 Invalid sensor filtering

Three tests verify that the assignment sentinel $[-10000, -10000, -10000]^T$ is discarded with its paired frames, a reading at exactly 5 mm remains valid, and missing or nonfinite readings (`None`, NaN, or infinity) are ignored. Valid observations still recover the known tip after unusable records are appended.

B.5 Minimum valid observation count

Zero or one valid reading is rejected, while two valid readings return the calibrated tip and a 3×3 covariance. One proper rotation is mathematically sufficient to determine the three tip coordinates; the two-observation minimum is imposed to estimate empirical covariance. Invalid sensor records do not count toward this minimum.

B.6 Frame-error propagation into tip calibration

The observation count is fixed at 15, and the sensor readings remain exact. Figure 10 reports means and sample standard deviations across 20 seeds while perturbing the pointer and calibration frames independently. These trials begin with frame estimates; marker refinement and SVD registration are not rerun in this study.

B.6.1 Frame-rotation-error propagation

Increasing the per-axis rotation standard deviation of the left-multiplied perturbation from 0.10° to 1.00° increases the mean tip error from 0.38781 to 3.91504 mm (left panel). The explicit translation component of the perturbation is zero.

B.6.2 Frame-translation-error propagation

With identity perturbation rotation, increasing per-axis translation standard deviation from 0.10 to 1.00 mm increases mean tip error from 0.05292 to 0.52916 mm (right panel). The two sweeps use different perturbation units and ranges; they do not provide a dimensionless ranking of rotation and translation sensitivity.

The implementation uses $F' = \Delta F F$, so $R' = R_\Delta R$ and $\boldsymbol{t}' = R_\Delta \boldsymbol{t} + \boldsymbol{t}_\Delta$. Consequently, the rotation sweep also changes frame-origin coordinates even when $\boldsymbol{t}_\Delta = 0$. Its result includes this coordinate-dependent effect and should not be described as a rotation-only change at fixed translation or attributed solely to the pointer-tip lever arm.

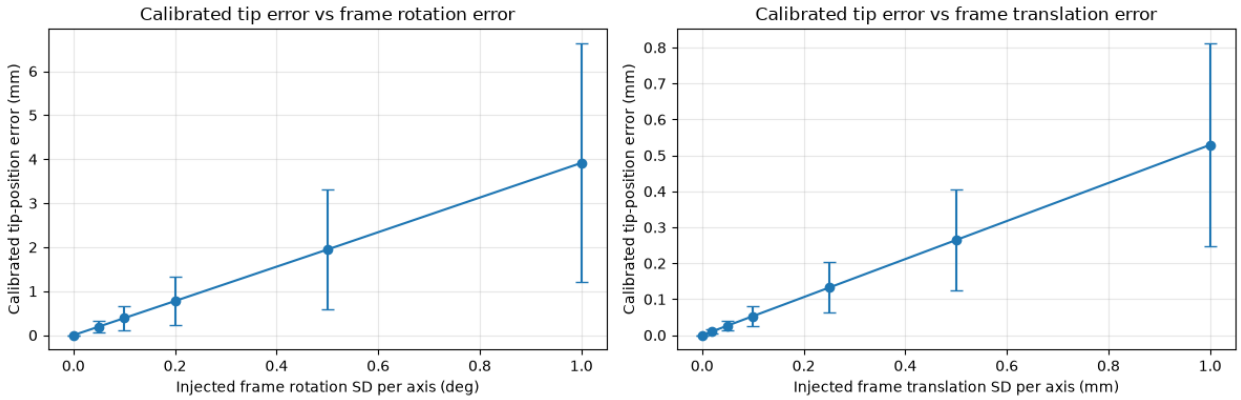


Figure 10: Frame-error propagation into tip calibration (B.6).

B.7 Covariance calculation

Two analytic tests check the covariance calculation. Identical single-observation tip estimates give a zero covariance matrix. Estimates with x offsets of -1, 0, and $+1$ mm have sample variance $1\,\mathrm{mm}^2$; the covariance of their three-sample mean is therefore

$$\widehat{C}_{\mathrm{tip}} = \mathrm{diag}(1/3, 0, 0)\ \mathrm{mm}^2. \tag{94}$$

The computed matrix agrees with this analytic result.

5.4 Limitations

All deterministic tests pass, but the sensitivity results apply only to the stated seed sets and the result for the 102 poses within 45° is one simulation realization. Empirical tip covariance also excludes marker-model bias shared across observations. The reported pass rates are therefore empirical results, not guarantees for every admissible pose.

6 Result

6.1 Marker-refinement result

Refinement updates the nominal local marker coordinates before calibration and testing. Table 16 reports the magnitude $\left\| \widehat{\boldsymbol{a}}_{m,k}^{(N_r)} - \boldsymbol{a}_{m,k} \right\|_2$ for each marker. These values are model updates, not errors relative to hidden ground truth.

Table 16: Marker-coordinate updates after refinement [mm].

Body	M1	M2	M3	M4	M5	Mean
Pointer	0.939	1.803	0.714	0.924	1.456	1.167
Calibration	0.652	0.435	0.536	0.812	0.588	0.605
Nearby	0.863	1.489	1.147	1.167	0.944	1.122

6.2 Calibrated-tip result

From 60 valid observations, calibration produces the tip coordinate in the pointer frame:

$$\widehat{\boldsymbol{p}}_{\text{tip}} = [1.362, -4.432, -149.684]^T \text{ mm}. \tag{95}$$

Its estimated covariance and per-axis standard deviation are

$$\widehat{C}_{\text{tip}} = \begin{bmatrix} 0.001 & 0 & 0.001 \\ 0 & 0.002 & 0 \\ 0.001 & 0 & 0.002 \end{bmatrix} \text{ mm}^2, \qquad \boldsymbol{\sigma}_{\text{tip}} = [0.037, 0.047, 0.047]^T \text{ mm}. \tag{96}$$

This pointer-frame vector is the calibrated input used by Step 8; it is not a single position representing all test poses.

6.3 Step 8 test result

For test pose i, the program predicts $\widehat{\boldsymbol{p}}_{t,\text{near},i}$ in the nearby frame. All reported poses use Euler-angle components from $\Theta = \{0, \pm\pi/6, \pm\pi/3, \pm\pi/2\}$ and have an actual pointer-axis tilt of at most 45°. There are $N_{45} = 102$ such poses,

$$\Delta\boldsymbol{p}_i = \widehat{\boldsymbol{p}}_{t,\text{near},i} - \boldsymbol{p}^*_{t,\text{near},i}, \tag{97}$$

and the empirical mean and sample covariance are

$$\overline{\Delta\boldsymbol{p}}_{45} = [0.044, -0.546, 0.998]^T \text{ mm}, \qquad \widehat{C}_{\Delta p,45} = \begin{bmatrix} 0.203 & -0.036 & -0.049 \\ -0.036 & 0.170 & -0.109 \\ -0.049 & -0.109 & 0.100 \end{bmatrix} \text{ mm}^2. \tag{98}$$

Table 17: Nearby-frame tip errors within 45° of vertical.

N_{45}	Mean [mm]	RMSE [mm]	Maximum [mm]	Within 2 mm
102	1.260	1.329	2.155	96.1%

The assignment-required region achieves a 96.1% empirical pass rate. In the nearby-size comparison, the 500 mm candidate gives an RMSE of 1.022 mm, a maximum error of 1.890 mm, and a 100% pass rate over these poses.

7 AI Assistance and Use of Tools

AI assistance was used to check code and support mathematical derivations; OpenAI Codex also assisted with notebook comparison and report preparation. The prompts requested code checks

and help with formula derivations, a block-by-block comparison with the unfinished notebook, and advice on placing the unit-test plots in the report. Follow-up prompts requested insertion of the plots with captions, A.1–A.5 and B.1–B.7 headings, consistent table fonts, a shorter repository reference, correction of Figure 4, and a full formatting review. AI-assisted explanatory text and LaTeX edits were incorporated into the report; the numerical plot data came from saved notebook outputs, and LaTeX/Tectonic was used to compile and inspect the PDF.

References

[1] R. H. Taylor, *Assignment 1 – CIS I (601.455/655), Fall 2026*, September 24, 2026, pp. 2–7, `2026_assignment_1_.pdf`.

[2] R. H. Taylor, *Calibration*, CIS I lecture notes, Fall 2026, slides 7–15.

[3] R. H. Taylor, *Linear Least Squares Review*, CIS I lecture notes, Fall 2026, slides 1–10.

[4] LCSR-CIIS, *CIS I Course repository*. `https://github.com/LCSR-CIIS/CIS_I_Course`.

[5] Northern Digital Inc., *NDI Passive Spheres*, 11.8-mm passive marker sphere specification, accessed October 10, 2026. `https://www.ndigital.com/polaris/ndi-passive-sphere/`.

[6] K. Rácz, B. Seregély, and R. M. Kiss, "Performance of optically tracked instrumented pointers: a systematic literature review," *Biomedical Engineering Online*, vol. 24, art. 142, 2025. `https://pmc.ncbi.nlm.nih.gov/articles/PMC12676873/`.

[7] A. Manbachi, J. H. Siewerdsen, et al., "Design and validation of an open-source library of dynamic reference frames for research and education in optical tracking," *Journal of Medical Imaging*, vol. 5, no. 2, 2018. `https://pmc.ncbi.nlm.nih.gov/articles/PMC5806031/`.